

# walmart-business-case-study

April 2, 2024

## 1 *Business Case: Walmart - Confidence Interval and CLT*

### **Objective :**

The key objective is to understand and analyze customer purchase behavior, specifically focusing on the purchase amount, with respect to customers' gender and other relevant factors. The key objective is to determine if there are differences in spending habits between male and female customers, particularly on Black Friday.

### **About Data :**

User\_ID: This column represents the unique identifier for each customer (User ID). It helps in tracking individual customer transactions and behaviors.

Product\_ID: This column contains the unique identifier for each product (Product ID) purchased by customers. It helps in categorizing and analyzing sales data based on specific products.

Gender: This column indicates the gender (sex) of the customer, with possible values being "Male" or "Female". It allows for gender-based analysis of purchase behavior.

Age: The "Age" column represents the age of customers categorized into bins or age groups. For example, age groups could be "Under 18", "18-25", "26-35", "36-45", "46-50", and "51-55", among others.

Occupation: This column contains information about the occupation of customers, although it may be masked or anonymized for privacy reasons. Occupation data can provide insights into the purchasing power and preferences of different occupational groups.

City\_Category: The "City\_Category" column categorizes customers based on the city they reside in, with categories typically denoted as "A", "B", or "C". This information helps in understanding regional variations in purchase behavior and preferences.

StayInCurrentCityYears: This column indicates the number of years a customer has stayed in their current city. It may provide insights into customer loyalty, stability, and potential long-term purchasing behavior.

Marital\_Status: The "Marital\_Status" column indicates the marital status of customers, often represented as binary values such as "0" for unmarried and "1" for married. Marital status can influence purchase decisions, especially for products related to households and families.

ProductCategory: This column contains information about the product categories purchased by customers. Similar to occupation, this data may be masked or anonymized to protect customer privacy.

Purchase: The “Purchase” column represents the purchase amount (in currency units) made by each customer for the respective product. It is a numerical variable and forms the basis for analyzing purchase behavior and spending patterns.

This dataset can be used for various analyses and insights, such as:

Analyzing gender-based spending patterns on Black Friday. Understanding the impact of age groups on purchase behavior. Investigating differences in purchasing habits across city categories. Exploring the relationship between marital status and purchase amounts. Conducting product-level analysis to identify top-selling products and categories. Predictive modeling to forecast sales and customer behavior.

```
[271]: # Import the required library and dataset
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import norm
```

```
[272]: df = pd.read_csv('Wallmart_case_study_data.csv')
df.head()
```

```
[272]:
```

|   | User_ID | Product_ID | Gender | Age  | Occupation | City_Category | \ |
|---|---------|------------|--------|------|------------|---------------|---|
| 0 | 1000001 | P00069042  | F      | 0-17 | 10         | A             |   |
| 1 | 1000001 | P00248942  | F      | 0-17 | 10         | A             |   |
| 2 | 1000001 | P00087842  | F      | 0-17 | 10         | A             |   |
| 3 | 1000001 | P00085442  | F      | 0-17 | 10         | A             |   |
| 4 | 1000002 | P00285442  | M      | 55+  | 16         | C             |   |

|   | Stay_In_Current_City_Years | Marital_Status | Product_Category | Purchase |
|---|----------------------------|----------------|------------------|----------|
| 0 | 2                          | 0              | 3                | 8370     |
| 1 | 2                          | 0              | 1                | 15200    |
| 2 | 2                          | 0              | 12               | 1422     |
| 3 | 2                          | 0              | 12               | 1057     |
| 4 | 4+                         | 0              | 8                | 7969     |

```
[273]: df.tail()
```

```
[273]:
```

|        | User_ID | Product_ID | Gender | Age   | Occupation | City_Category | \ |
|--------|---------|------------|--------|-------|------------|---------------|---|
| 450771 | 1003469 | P00028042  | M      | 46-50 | 17         | B             |   |
| 450772 | 1003469 | P00220242  | M      | 46-50 | 17         | B             |   |
| 450773 | 1003469 | P00004842  | M      | 46-50 | 17         | B             |   |
| 450774 | 1003469 | P00057542  | M      | 46-50 | 17         | B             |   |
| 450775 | 1003469 | P00101242  | M      | 46-50 | 17         | B             |   |

|        | Stay_In_Current_City_Years | Marital_Status | Product_Category | Purchase |
|--------|----------------------------|----------------|------------------|----------|
| 450771 | 3                          | 1              | 1                | 3939     |
| 450772 | 3                          | 1              | 3                | 11073    |
| 450773 | 3                          | 1              | 3                | 8137     |

|        |   |   |   |      |
|--------|---|---|---|------|
| 450774 | 3 | 1 | 3 | 8254 |
| 450775 | 3 | 1 | 5 | 8626 |

Performing typical data analysis such as

1. Structure & characteristics of the dataset
2. Detecting Null values & Outliers
3. Handling Null values and outliers
4. Checking the difference between mean and median

```
[274]: df.shape
```

```
[274]: (450776, 10)
```

```
[275]: df.columns
```

```
[275]: Index(['User_ID', 'Product_ID', 'Gender', 'Age', 'Occupation', 'City_Category',
          'Stay_In_Current_City_Years', 'Marital_Status', 'Product_Category',
          'Purchase'],
          dtype='object')
```

```
[276]: df.dtypes
```

```
[276]: User_ID          int64
Product_ID         object
Gender             object
Age               object
Occupation         int64
City_Category      object
Stay_In_Current_City_Years  object
Marital_Status     int64
Product_Category   int64
Purchase           int64
dtype: object
```

```
[277]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 450776 entries, 0 to 450775
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   User_ID               450776 non-null  int64
1   Product_ID            450776 non-null  object
2   Gender                450776 non-null  object
3   Age                   450776 non-null  object
4   Occupation            450776 non-null  int64
5   City_Category         450776 non-null  object
```

```

6 Stay_In_Current_City_Years 450776 non-null object
7 Marital_Status              450776 non-null int64
8 Product_Category            450776 non-null int64
9 Purchase                    450776 non-null int64
dtypes: int64(5), object(5)
memory usage: 34.4+ MB

```

## Insights

From the above analysis, it is clear that, data has total of 10 features and 122,293 rows with lots of mixed alpha numeric data.

The dataset has a mix of categorical and numerical data.

Missing value in the “Gender” column requires handling.

Data types seem appropriate for analysis.

Suitable for analyzing customer purchase behavior, segmentation, and insights generation.

```

[278]: def missingValue(df):
        #Identifying Missing data. Already verified above. To be sure again
        ↪checking.
        total_null = df.isnull().sum().sort_values(ascending = False)
        percent = ((df.isnull().sum()/df.isnull().count())*100).
        ↪sort_values(ascending = False)
        print("Total records = ", df.shape[0])

        md = pd.concat([total_null,percent.round(2)],axis=1,keys=['Total
        ↪Missing','In Percent'])
        return md

```

```

[279]: missingValue(df).head(5)

```

```
Total records = 450776
```

```

[279]:
      Total Missing  In Percent
User_ID           0          0.0
Product_ID        0          0.0
Gender            0          0.0
Age              0          0.0
Occupation        0          0.0

```

```

[280]: np.any(df.isna())

```

```
[280]: False
```

```

[281]: np.any(df.duplicated())

```

```
[281]: False
```

## 2 Memory Optimization

```
[282]: # Converting User_ID column datatype to int32
df['User_ID'] = df['User_ID'].astype('int32')

[283]: # Updating 'Marital_Status' column
df['Marital_Status'] = df['Marital_Status'].apply(lambda x: 'Married' if x == 1_
↪else 'Single')
df['Marital_Status'] = df['Marital_Status'].astype('category')

[284]: # Converting 'Age' column datatype to category
df['Age'] = df['Age'].astype('category')

[285]: # Converting 'Product_Category' column datatype to int8
df['Product_Category'] = df['Product_Category'].astype('int8')

[286]: # Converting 'Occupation' column's datatype to int8
df['Occupation'] = df['Occupation'].astype('int8')

[287]: # Converting 'City_Category' column's datatype to category
df['City_Category'] = df['City_Category'].astype('category')

[288]: # Converting 'Stay_In_Current_City_Years' column's datatype to category
df['Stay_In_Current_City_Years'] = df['Stay_In_Current_City_Years'].
↪astype('category')

[289]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 450776 entries, 0 to 450775
Data columns (total 10 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   User_ID                               450776 non-null  int32
1   Product_ID                           450776 non-null  object
2   Gender                                450776 non-null  object
3   Age                                   450776 non-null  category
4   Occupation                            450776 non-null  int8
5   City_Category                         450776 non-null  category
6   Stay_In_Current_City_Years           450776 non-null  category
7   Marital_Status                        450776 non-null  category
8   Product_Category                      450776 non-null  int8
9   Purchase                              450776 non-null  int64
dtypes: category(4), int32(1), int64(1), int8(2), object(2)
memory usage: 14.6+ MB
```

```
[290]: # For measurable quantities
df.describe()
```

```
[290]:
```

|       | User_ID      | Occupation    | Product_Category | Purchase      |
|-------|--------------|---------------|------------------|---------------|
| count | 4.507760e+05 | 450776.000000 | 450776.000000    | 450776.000000 |
| mean  | 1.002964e+06 | 8.083789      | 5.298042         | 9328.934058   |
| std   | 1.721898e+03 | 6.523638      | 3.748306         | 4978.060158   |
| min   | 1.000001e+06 | 0.000000      | 1.000000         | 185.000000    |
| 25%   | 1.001457e+06 | 2.000000      | 1.000000         | 5866.000000   |
| 50%   | 1.002967e+06 | 7.000000      | 5.000000         | 8061.000000   |
| 75%   | 1.004408e+06 | 14.000000     | 8.000000         | 12069.000000  |
| max   | 1.006040e+06 | 20.000000     | 18.000000        | 23961.000000  |

The dataset provides information on the following variables:

User\_ID: It contains unique identification numbers assigned to each user. The dataset includes a total of 450,776 user records.

Occupation: This variable represents the occupation of the users. The dataset includes values ranging from 0 to 20, indicating different occupations.

Product\_Category: It indicates the category of the products purchased by the users. The dataset includes values ranging from 1 to 18, representing different product categories.

Purchase: This variable represents the purchase amount made by each user. The dataset includes purchase values ranging from 185 to 23,961.

```
[291]: # description of columns with 'object' datatype
df.describe(include = 'object')
```

```
[291]:
```

|        | Product_ID | Gender |
|--------|------------|--------|
| count  | 450776     | 450776 |
| unique | 3599       | 2      |
| top    | P00265242  | M      |
| freq   | 1530       | 339632 |

The provided data represents summary statistics for two variables: Product\_ID and Gender. Here is a breakdown of the information:

Product\_ID: There are 3,599 unique values observed in this variable, indicating that there are 3,599 different products. The top value, which appears most frequently, is 'P00265242'. This value occurs 1,530 times in the dataset.

Gender: There are 2 unique values in this variable, which suggests that it represents a binary category. The top value is 'M', indicating that 'M' is the most common gender category. It appears 339,632 times in the dataset.

These summary statistics provide insights into the distribution and frequency of the Product\_ID and Gender variables. They give an understanding of the number of unique products, the most common product, and the dominant gender category in the dataset.

```
[292]: # How many unique customers' data is given in the dataset?  
df['User_ID'].nunique()
```

```
[292]: 5891
```

We have the data of 5891 customers who made at least one purchase on Black Friday in Walmart.

```
[293]: # Total number of transactions made by each gender  
np.round(df['Gender'].value_counts(normalize = True) * 100, 2)
```

```
[293]: M    75.34  
      F    24.66  
      Name: Gender, dtype: float64
```

It is clear from the above that out of every four transactions, three are made by males.

```
[294]: np.round(df['Occupation'].value_counts(normalize = True) * 100, 2).cumsum()
```

```
[294]: 4    13.23  
      0    25.87  
      7    36.59  
      1    45.16  
      17   52.41  
      20   58.57  
      12   64.24  
      14   69.20  
      2    74.01  
      16   78.61  
      6    82.29  
      3    85.52  
      10   87.90  
      5    90.09  
      15   92.27  
      11   94.40  
      19   95.95  
      13   97.38  
      18   98.56  
      9    99.71  
      8    99.99  
      Name: Occupation, dtype: float64
```

```
[295]: np.round(df['Stay_In_Current_City_Years'].value_counts(normalize = True) * 100, 2)  
      ↪2)
```

```
[295]: 1    35.17  
      2    18.48  
      3    17.38  
      4+   15.45
```

```
0      13.51
Name: Stay_In_Current_City_Years, dtype: float64
```

From the above result, it is clear that majority of the transactions (53.75 % of total transactions) are made by the customers having 1 or 2 years of stay in the current city.

```
[296]: np.round(df['Product_Category'].value_counts(normalize = True).head(10) * 100, 2).cumsum()
```

```
[296]: 5      27.66
      1      53.33
      8      74.23
     11      78.67
      2      83.04
      6      86.78
      3      90.47
      4      92.64
     16      94.45
     15      95.61
Name: Product_Category, dtype: float64
```

It can be inferred from the above result that 82.43% of the total transactions are made for only 5 Product Categories. These are, 5, 1, 8, 11 and 2.

```
[297]: # How many unique customers are there for each gender
df_gender_dist = pd.DataFrame(df.groupby(by = ['Gender'])['User_ID'].nunique()).
    reset_index().rename(columns = {'User_ID' : 'unique_customers'})
df_gender_dist['percent_share'] = np.round(df_gender_dist['unique_customers'] /
    df_gender_dist['unique_customers'].sum() * 100, 2)
df_gender_dist
```

```
[297]:   Gender  unique_customers  percent_share
0      F              1666           28.28
1      M              4225           71.72
```

```
[298]: # How many transactions are made by each gender category ?
df.groupby(by = ['Gender'])['User_ID'].count()
```

```
[298]: Gender
      F    111144
      M    339632
Name: User_ID, dtype: int64
```

```
[299]: print('Average number of transactions made by each Male on Black Friday is',
    round(414259 / 4225))
print('Average number of transactions made by each Female on Black Friday is',
    round(135809 / 1666))
```



Average number of transactions made by each Male on Black Friday is 98  
 Average number of transactions made by each Female on Black Friday is 82

```
[300]: # What is the total Revenue generated by Walmart from each Gender ?
df_gender_revenue = df.groupby(by = ['Gender'])['Purchase'].sum().to_frame().
    ↪sort_values(by = 'Purchase', ascending = False).reset_index()
df_gender_revenue['percent_share'] = np.round((df_gender_revenue['Purchase'] /
    ↪df_gender_revenue['Purchase'].sum()) * 100, 2)
df_gender_revenue
```

```
[300]:   Gender   Purchase  percent_share
0      M  3226861693           76.73
1      F   978397886           23.27
```

```
[301]: # What is the average total purchase made by each user in each gender ?
df1 = pd.DataFrame(df.groupby(by = ['Gender', 'User_ID'])['Purchase'].sum()).
    ↪reset_index().rename(columns = {'Purchase' : 'Average_Purchase'})
df1.groupby(by = 'Gender')['Average_Purchase'].mean()
```

```
[301]: Gender
F      587273.641056
M      763754.246864
Name: Average_Purchase, dtype: float64
```

```
[302]: # What is the Average Revenue generated by Walmart from each Gender per
    ↪transaction ?
pd.DataFrame(df.groupby(by = 'Gender')['Purchase'].mean()).reset_index().
    ↪rename(columns = {'Purchase' : 'Average_Purchase'})
```

```
[302]:   Gender  Average_Purchase
0      F      8802.975293
1      M      9501.053178
```

```
[303]: from scipy import stats

filtered_gender = df[['Gender', 'Purchase']]

mean_female = filtered_gender[filtered_gender['Gender'] == 'F']['Purchase'].
    ↪mean()
mean_male = filtered_gender[filtered_gender['Gender'] == 'M']['Purchase'].mean()

std_female = filtered_gender[filtered_gender['Gender'] == 'F']['Purchase'].std()
std_male = filtered_gender[filtered_gender['Gender'] == 'M']['Purchase'].std()

# Sample size (you can change this value to observe different distributions)
n = 1000
```

```

# Confidence level (you can change this to 90%, 95%, or 99%)
confidence_level = 0.95

# Calculate the z-score based on the confidence level
z_score = stats.norm.ppf((1 + confidence_level) / 2)

# Calculate the margin of error
margin_of_error_female = z_score * (std_female / np.sqrt(n))
margin_of_error_male = z_score * (std_male / np.sqrt(n))

# Calculate confidence intervals
ci_female_lower = mean_female - margin_of_error_female
ci_female_upper = mean_female + margin_of_error_female

ci_male_lower = mean_male - margin_of_error_male
ci_male_upper = mean_male + margin_of_error_male

print(f"Confidence Interval for Female Customers: ({ci_female_lower},
↳{ci_female_upper})")
print(f"Confidence Interval for Male Customers: ({ci_male_lower},
↳{ci_male_upper})")

```

Confidence Interval for Female Customers: (8510.64369198775, 9095.306894638607)  
Confidence Interval for Male Customers: (9188.130074714027, 9813.976281577488)

From these intervals, we can observe that the confidence intervals for male and female customers do overlap. Specifically, the lower limit of the male customer spending interval (9188.13) overlaps with the upper limit of the female customer spending interval (9095.31). This overlapping indicates that there is no statistically significant difference between the average spending of male and female customers at the chosen confidence level (95%).

```

[304]: # How many unique customers are there for each Marital Status ?
df_marital_status_dist = pd.DataFrame(df.groupby(by =
↳['Marital_Status'])['User_ID'].nunique()).reset_index().rename(columns =
↳{'User_ID' : 'unique_customers'})
df_marital_status_dist['percent_share'] = np.
↳round(df_marital_status_dist['unique_customers'] /
↳df_marital_status_dist['unique_customers'].sum() * 100, 2)
df_marital_status_dist

```

```

[304]:   Marital_Status  unique_customers  percent_share
0         Married           2474           42.0
1         Single           3417           58.0

```

```

[305]: # How many transactions are made by each Marital Status category ?
df.groupby(by = ['Marital_Status'])['User_ID'].count()

```

```
[305]: Marital_Status
Married    184509
Single     266267
Name: User_ID, dtype: int64
```

```
[306]: print('Average number of transactions made by each user with marital status_
↳Married is', round(225337 / 2474))
print('Average number of transactions made by each with marital status Single_
↳is', round(324731 / 3417))
```

Average number of transactions made by each user with marital status Married is 91

Average number of transactions made by each with marital status Single is 95

```
[307]: # What is the total Revenue generated by Walmart from each Marital Status ?
df_marital_status_revenue = df.groupby(by = ['Marital_Status'])['Purchase'].
↳sum().to_frame().sort_values(by = 'Purchase', ascending = False).
↳reset_index()
df_marital_status_revenue['percent_share'] = np.
↳round((df_marital_status_revenue['Purchase'] /_
↳df_marital_status_revenue['Purchase'].sum()) * 100, 2)
df_marital_status_revenue
```

```
[307]: Marital_Status    Purchase    percent_share
0         Single    2483210061         59.05
1         Married    1722049518         40.95
```

```
[381]: # What is the average total purchase made by each user in each marital status ?
df1 = pd.DataFrame(df.groupby(by = ['Marital_Status', 'User_ID'])['Purchase'].
↳sum()).reset_index().rename(columns = {'Purchase' : 'Average_Purchase'})
df_married = df1.groupby(by = 'Marital_Status')['Average_Purchase'].mean()
df_married
```

```
[381]: Marital_Status
Married    292318.709557
Single     421526.067051
Name: Average_Purchase, dtype: float64
```

```
[383]: from scipy import stats

n = 1000

# Confidence level (you can change this to 90%, 95%, or 99%)
confidence_level = 0.95

# Calculate the z-score based on the confidence level
z_score = stats.norm.ppf((1 + confidence_level) / 2)
```

```

# Calculate mean and standard deviation for married and single customers
mean_married = df_married['Married']
mean_single = df_married['Single']

std_married = df1[df1['Marital_Status'] == 'Married']['Average_Purchase'].std()
std_single = df1[df1['Marital_Status'] == 'Single']['Average_Purchase'].std()

# Calculate the margin of error
margin_of_error_married = z_score * (std_married / np.sqrt(n))
margin_of_error_single = z_score * (std_single / np.sqrt(n))

# Calculate confidence intervals
ci_married_lower = mean_married - margin_of_error_married
ci_married_upper = mean_married + margin_of_error_married

ci_single_lower = mean_single - margin_of_error_single
ci_single_upper = mean_single + margin_of_error_single

print(f"Confidence Interval for Married Customers: ({ci_married_lower},
↪{ci_married_upper})")
print(f"Confidence Interval for Single Customers: ({ci_single_lower},
↪{ci_single_upper})")

```

Confidence Interval for Married Customers: (254653.00975816516,  
329984.40935573744)

Confidence Interval for Single Customers: (378224.7487282499, 464827.3853746188)

The confidence intervals reveal a substantial spending disparity between married and single customers, with single customers exhibiting significantly higher average purchase amounts. The estimated average purchase amount for married customers falls between \$254,653.01 and \$329,984.41, while for single customers, it ranges from \$378,224.75 to \$464,827.39, both with 95% confidence. This insight suggests a need for tailored marketing strategies and product offerings by Walmart, leveraging targeted promotions and personalized engagement to capitalize on the distinct spending behaviors of these customer segments and ultimately drive business growth and customer satisfaction.

```

[309]: df_age_dist = pd.DataFrame(df.groupby(by = ['Age'])['User_ID'].nunique()).
↪reset_index().rename(columns = {'User_ID' : 'unique_customers'}).
↪sort_values(by = 'unique_customers', ascending = False)
df_age_dist['percent_share'] = np.round(df_age_dist['unique_customers'] /
↪df_age_dist['unique_customers'].sum() * 100, 2)
df_age_dist['cumulative_percent'] = df_age_dist['percent_share'].cumsum()
df_age_dist

```

```

[309]:   Age  unique_customers  percent_share  cumulative_percent
2  26-35                2053           34.85             34.85
3  36-45                1167           19.81             54.66

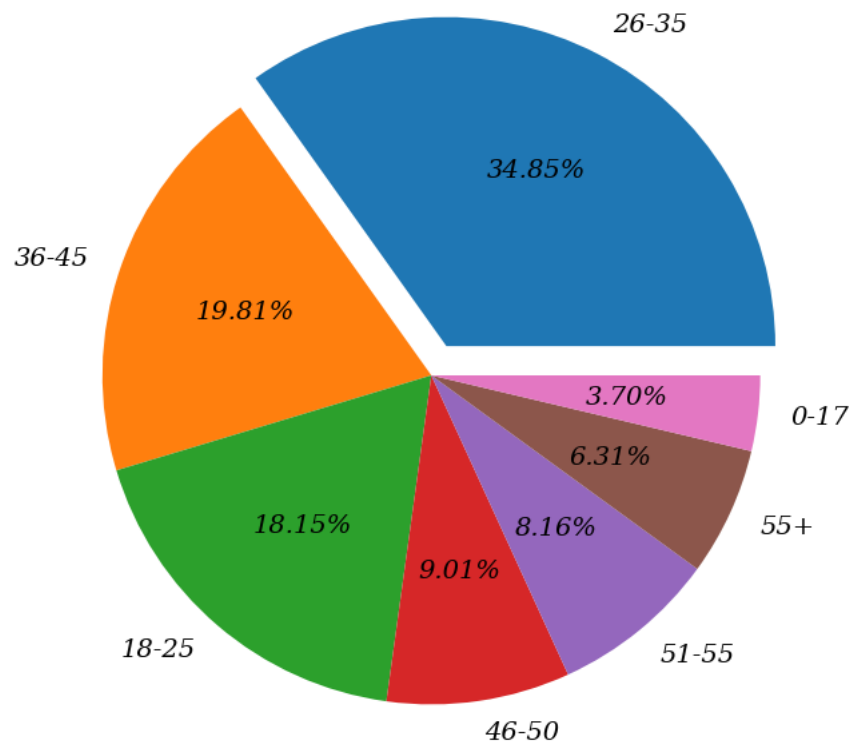
```

|   |       |      |       |       |
|---|-------|------|-------|-------|
| 1 | 18-25 | 1069 | 18.15 | 72.81 |
| 4 | 46-50 | 531  | 9.01  | 81.82 |
| 5 | 51-55 | 481  | 8.16  | 89.98 |
| 6 | 55+   | 372  | 6.31  | 96.29 |
| 0 | 0-17  | 218  | 3.70  | 99.99 |

```
[310]: plt.figure(figsize = (8, 8))
plt.title('Share of Unique customers based on their age group', fontdict = {
    'fontsize' : 20,
    'fontstyle' : 'oblique',
    'fontfamily' : 'serif',
    'fontweight' : 600} )
plt.pie(x = df_age_dist['percent_share'], labels = df_age_dist['Age'],
    explode = [0.1] + [0] * 6, autopct = '%.2f%%',
    textprops = {'fontsize' : 14,
    'fontstyle' : 'oblique',
    'fontfamily' : 'serif',
    'fontweight' : 500})
plt.plot()
```

[310]: []

### ***Share of Unique customers based on their age group***



```
[311]: df['Age'].value_counts()
```

```
[311]: 26-35    179505
      36-45    90154
      18-25    82320
      46-50    37137
      51-55    31633
      55+     17551
      0-17    12476
      Name: Age, dtype: int64
```

```
[312]: df_age_revenue = pd.DataFrame(df.groupby(by = 'Age', as_index =
      ↪False)['Purchase'].sum()).sort_values(by = 'Purchase', ascending = False)
df_age_revenue['percent_share'] = np.round((df_age_revenue['Purchase'] /
      ↪df_age_revenue['Purchase'].sum()) * 100, 2)
df_age_revenue['cumulative_percent_share'] = df_age_revenue['percent_share'].
      ↪cumsum()
df_age_revenue
```

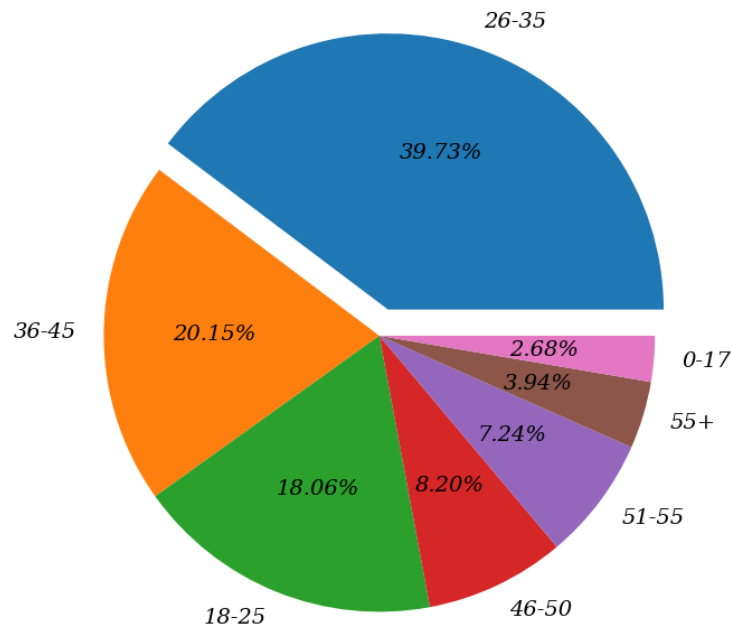
```
[312]:
```

|   | Age   | Purchase   | percent_share | cumulative_percent_share |
|---|-------|------------|---------------|--------------------------|
| 2 | 26-35 | 1670841254 | 39.73         | 39.73                    |
| 3 | 36-45 | 847175315  | 20.15         | 59.88                    |
| 1 | 18-25 | 759455557  | 18.06         | 77.94                    |
| 4 | 46-50 | 344688899  | 8.20          | 86.14                    |
| 5 | 51-55 | 304304920  | 7.24          | 93.38                    |
| 6 | 55+   | 165885706  | 3.94          | 97.32                    |
| 0 | 0-17  | 112907928  | 2.68          | 100.00                   |

```
[313]: plt.figure(figsize = (8, 8))
plt.title('Percentage share of revenue generated from each age category',
      ↪fontdict = {'fontsize' : 20,
                'fontstyle' : 'oblique',
                'fontfamily' : 'serif',
                'fontweight' : 600} )
plt.pie(x = df_age_revenue['percent_share'], labels = df_age_revenue['Age'],
      explode = [0.1] + [0] * 6, autopct = '%.2f%%',
      textprops = {'fontsize' : 14,
                    'fontstyle' : 'oblique',
                    'fontfamily' : 'serif',
                    'fontweight' : 500})
plt.plot()
```

```
[313]: []
```

### Percentage share of revenue generated from each age category



```
[314]: df_city_dist = pd.DataFrame(df.groupby(by = ['City_Category'])['User_ID'].
    ↪unique()).reset_index().rename(columns = {'User_ID' : 'unique_customers'})
df_city_dist['percent_share'] = np.round((df_city_dist['unique_customers'] /
    ↪df_city_dist['unique_customers'].sum()) * 100, 2)
df_city_dist['cumulative_percent_share'] = df_city_dist['percent_share'].
    ↪cumsum()
df_city_dist
```

```
[314]:
```

|   | City_Category | unique_customers | percent_share | cumulative_percent_share |
|---|---------------|------------------|---------------|--------------------------|
| 0 | A             | 1045             | 17.74         | 17.74                    |
| 1 | B             | 1707             | 28.98         | 46.72                    |
| 2 | C             | 3139             | 53.28         | 100.00                   |

```
[315]: df['City_Category'].value_counts()
```

```
[315]: B    189815
C     139489
A     121472
Name: City_Category, dtype: int64
```

```
[316]: # What is the revenue generated from different cities ?
df_city_revenue = df.groupby(by = ['City_Category'])['Purchase'].sum().
    ↪to_frame().sort_values(by = 'Purchase', ascending = False).reset_index()
```

```
df_city_revenue['percent_share'] = np.round((df_city_revenue['Purchase'] /
↳df_city_revenue['Purchase'].sum()) * 100, 2)
df_city_revenue['cumulative_percent_share'] = df_city_revenue['percent_share'].
↳cumsum()
df_city_revenue
```

```
[316]:
```

|   | City_Category | Purchase   | percent_share | cumulative_percent_share |
|---|---------------|------------|---------------|--------------------------|
| 0 | B             | 1745139096 | 41.50         | 41.50                    |
| 1 | C             | 1372557467 | 32.64         | 74.14                    |
| 2 | A             | 1087563016 | 25.86         | 100.00                   |

```
[317]: df.groupby(by = ['Product_Category'])['Product_ID'].nunique()
```

```
[317]:
```

| Product_Category |      |
|------------------|------|
| 1                | 490  |
| 2                | 151  |
| 3                | 90   |
| 4                | 88   |
| 5                | 964  |
| 6                | 119  |
| 7                | 100  |
| 8                | 1036 |
| 9                | 1    |
| 10               | 24   |
| 11               | 252  |
| 12               | 25   |
| 13               | 34   |
| 14               | 44   |
| 15               | 44   |
| 16               | 96   |
| 17               | 11   |
| 18               | 30   |

Name: Product\_ID, dtype: int64

```
[318]: # What is the revenue generated from different product categories ?
df_product_revenue = df.groupby(by = ['Product_Category'])['Purchase'].sum().
↳to_frame().sort_values(by = 'Purchase', ascending = False).reset_index()
df_product_revenue['percent_share'] = np.round((df_product_revenue['Purchase'] /
↳df_product_revenue['Purchase'].sum()) * 100, 2)
df_product_revenue['cumulative_percent_share'] =
↳df_product_revenue['percent_share'].cumsum()
df_product_revenue
```

```
[318]:
```

|   | Product_Category | Purchase   | percent_share | cumulative_percent_share |
|---|------------------|------------|---------------|--------------------------|
| 0 | 1                | 1573952116 | 37.43         | 37.43                    |
| 1 | 5                | 778023917  | 18.50         | 55.93                    |
| 2 | 8                | 706604400  | 16.80         | 72.73                    |



|    |    |           |      |        |
|----|----|-----------|------|--------|
| 3  | 6  | 267455671 | 6.36 | 79.09  |
| 4  | 2  | 221875238 | 5.28 | 84.37  |
| 5  | 3  | 168212369 | 4.00 | 88.37  |
| 6  | 16 | 120308324 | 2.86 | 91.23  |
| 7  | 11 | 93865559  | 2.23 | 93.46  |
| 8  | 10 | 83257073  | 1.98 | 95.44  |
| 9  | 15 | 76770751  | 1.83 | 97.27  |
| 10 | 7  | 50442634  | 1.20 | 98.47  |
| 11 | 4  | 22828797  | 0.54 | 99.01  |
| 12 | 14 | 16366828  | 0.39 | 99.40  |
| 13 | 18 | 7597490   | 0.18 | 99.58  |
| 14 | 9  | 5149347   | 0.12 | 99.70  |
| 15 | 17 | 4851203   | 0.12 | 99.82  |
| 16 | 12 | 4399567   | 0.10 | 99.92  |
| 17 | 13 | 3298295   | 0.08 | 100.00 |

```
[319]: top5 = df_product_revenue.head(5)['Purchase'].sum() / \
        ↪df_product_revenue['Purchase'].sum()
top5 = np.round(top5 * 100, 2)
print(f'Top 5 product categories from which Walmart makes {top5} % of total\
        ↪revenue are : {list(df_product_revenue["Product_Category"].head())}')

```

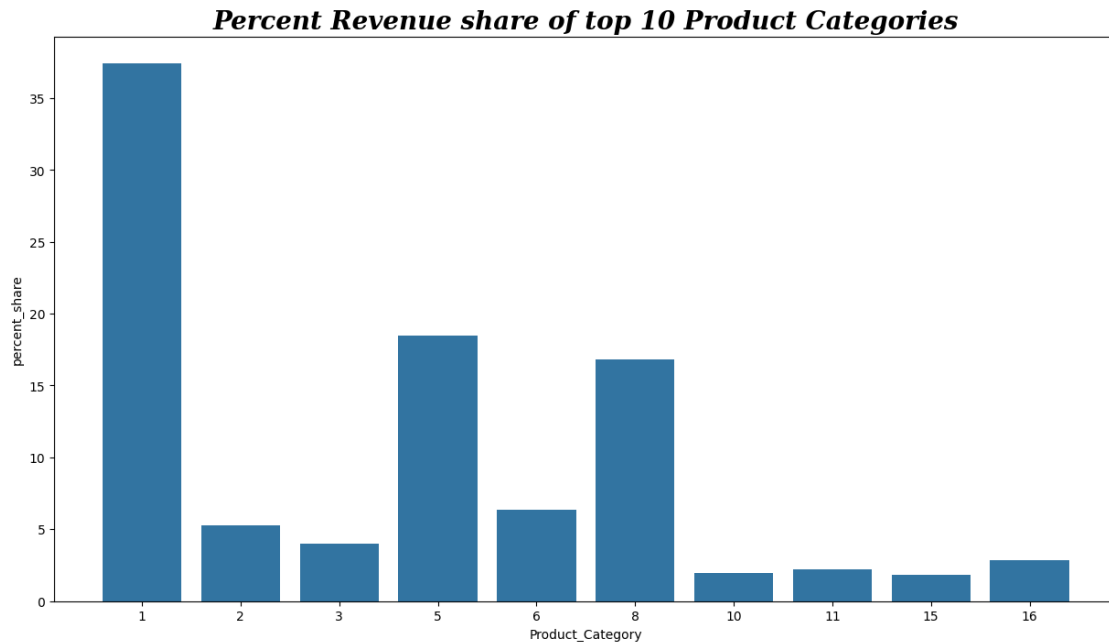
Top 5 product categories from which Walmart makes 84.37 % of total revenue are :  
[1, 5, 8, 6, 2]

```
[320]: plt.figure(figsize = (15, 8))
plt.title('Percent Revenue share of top 10 Product Categories', fontsize = 20,\
        ↪fontweight = 600, fontfamily = 'serif', fontstyle = 'oblique')
sns.barplot(data = df_product_revenue, x = df_product_revenue.\
        ↪head(10)['Product_Category'], y = df_product_revenue.\
        ↪head(10)['percent_share'])
plt.plot()

```

```
[320]: []

```



```
[321]: # What is the total Revenue generated by Walmart from each Gender ?
df_gender_revenue = df.groupby(by = ['Gender'])['Purchase'].sum().to_frame().
    ↪sort_values(by = 'Purchase', ascending = False).reset_index()
df_gender_revenue['percent_share'] = np.round((df_gender_revenue['Purchase'] /
    ↪df_gender_revenue['Purchase'].sum()) * 100, 2)
df_gender_revenue
```

```
[321]:   Gender  Purchase  percent_share
0      M  3226861693          76.73
1      F   978397886          23.27
```

```
[322]: # What is the Average Revenue generated by Walmart from each Gender per
    ↪transaction ?
pd.DataFrame(df.groupby(by = 'Gender')['Purchase'].mean()).reset_index().
    ↪rename(columns = {'Purchase' : 'Average_Purchase'})
```

```
[322]:   Gender  Average_Purchase
0      F      8802.975293
1      M      9501.053178
```

Distribution of number of Transactions :

```
[323]: plt.figure(figsize = (20, 10))
plt.suptitle('Distribution of number of Transactions Made', fontsize = 35,
    ↪fontweight = 600, fontfamily = 'serif', fontstyle = 'oblique')
plt.subplot(1, 3, 1)
```

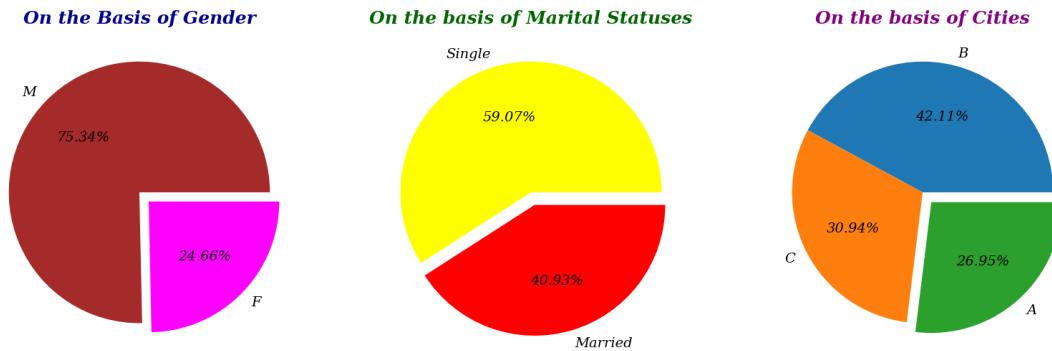
```

plt.title('On the Basis of Gender', color = 'darkblue', fontdict = {'fontsize' :
↪ 18,
                                'fontweight' : 600,
                                'fontstyle' : 'oblique',
                                'fontfamily' : 'serif'})
df_gender_dist = np.round(df['Gender'].value_counts(normalize = True) * 100, 2)
plt.pie(x = df_gender_dist.values, labels = df_gender_dist.index,
        explode = [0, 0.1], autopct = '%.2f%%',
        textprops = {'fontsize' : 14,
                      'fontstyle' : 'oblique',
                      'fontfamily' : 'serif',
                      'fontweight' : 500},
        colors = ['brown', 'magenta'])
plt.plot()
plt.subplot(1, 3, 2)
plt.title('On the basis of Marital Statuses', color = 'darkgreen', fontdict = ↵
↪ {'fontsize' : 18,
                                'fontweight' : 600,
                                'fontstyle' : 'oblique',
                                'fontfamily' : 'serif'})
df_Marital_Status_dist = np.round(df['Marital_Status'].value_counts(normalize = ↵
↪ True) * 100, 2)
plt.pie(x = df_Marital_Status_dist.values, labels = df_Marital_Status_dist.
↪ index,
        explode = [0, 0.1], autopct = '%.2f%%',
        textprops = {'fontsize' : 14,
                      'fontstyle' : 'oblique',
                      'fontfamily' : 'serif',
                      'fontweight' : 500},
        colors = ['yellow', 'red'])
plt.plot()
plt.subplot(1, 3, 3)
plt.title("On the basis of Cities", color = 'purple', fontdict = {'fontsize' : ↵
↪ 18,
                                'fontweight' : 555,
                                'fontstyle' : 'oblique',
                                'fontfamily' : 'serif'})
df_City_Category_dist = np.round(df['City_Category'].value_counts(normalize = ↵
↪ True) * 100, 2)
plt.pie(x = df_City_Category_dist.values, labels = df_City_Category_dist.index,
        explode = [0, 0, 0.1], autopct = '%.2f%%',
        textprops = {'fontsize' : 14,
                      'fontstyle' : 'oblique',
                      'fontfamily' : 'serif',
                      'fontweight' : 500})
plt.plot()

```

[323]: []

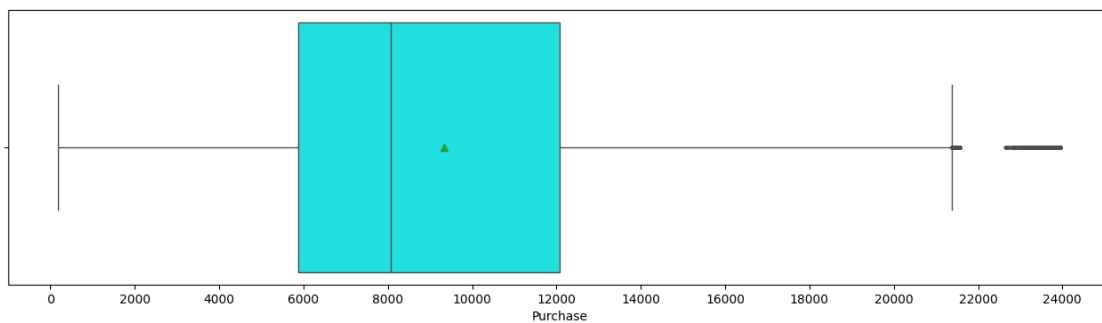
## ***Distribution of number of Transactions Made***



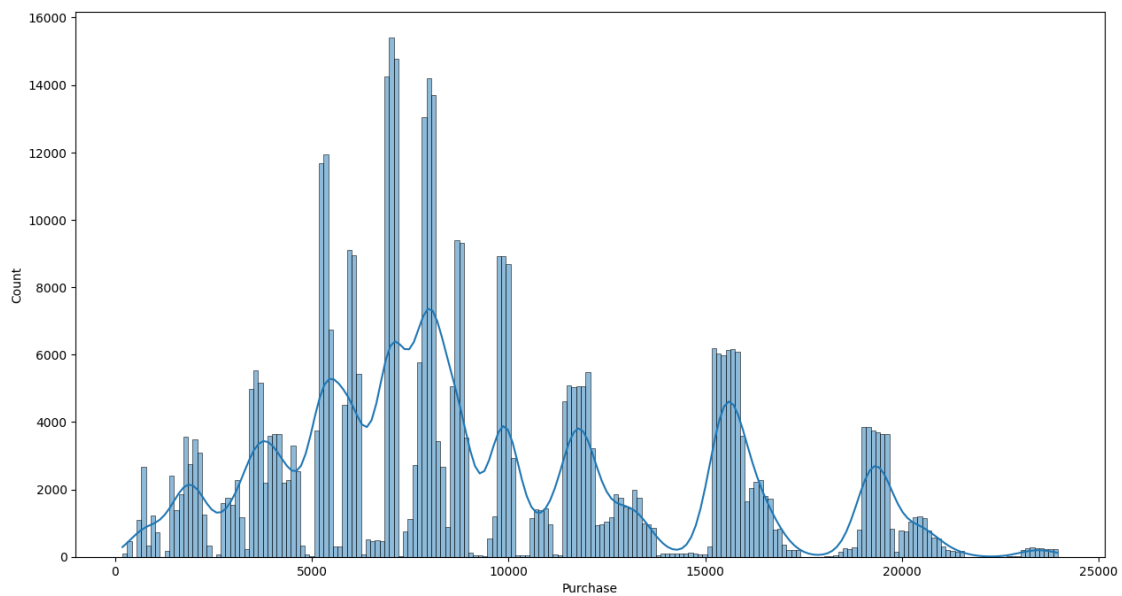
Univariate Analysis

```
[324]: plt.figure(figsize = (16, 4))
sns.boxplot(data = df,
            x = 'Purchase',
            showmeans = True,
            fliersize = 2,
            width = 0.5,
            color = np.random.choice(['magenta', 'lightgreen', 'cyan']))
plt.xticks(np.arange(0, 25001, 2000))
plt.plot()
```

[324]: []

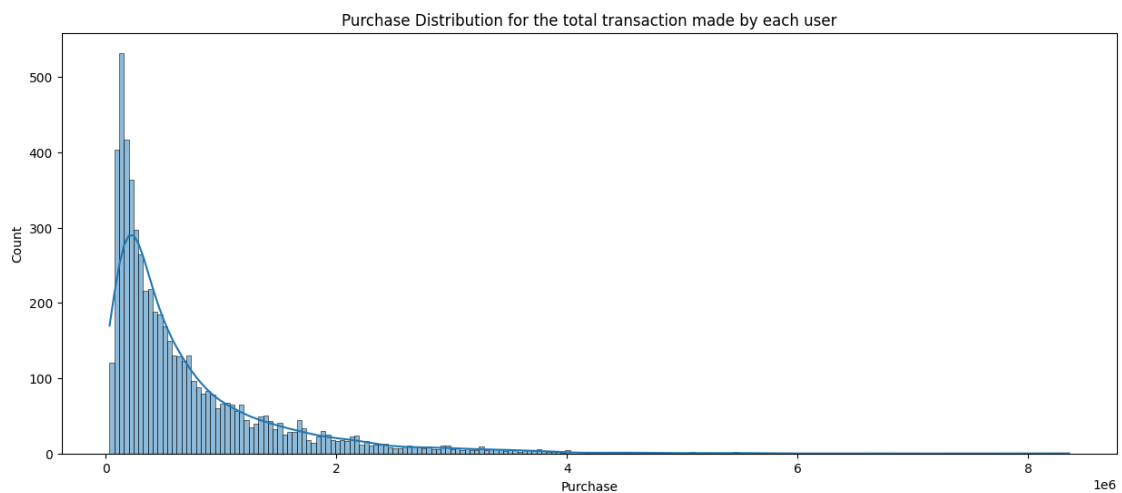


```
[325]: plt.figure(figsize = (15, 8))
sns.histplot(data = df, x = 'Purchase', kde = True, bins = 200)
plt.show()
```

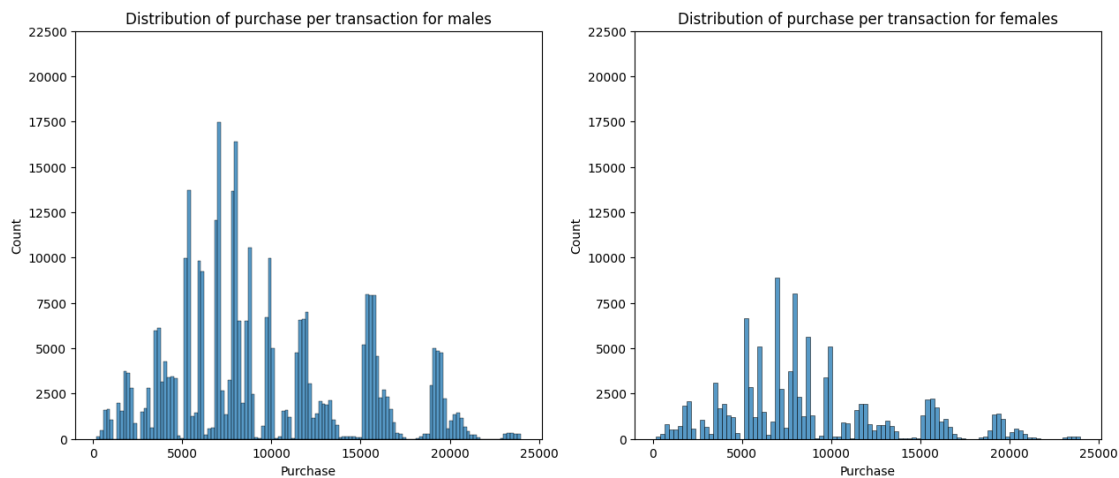


```
[326]: plt.figure(figsize = (15, 6))
plt.title('Purchase Distribution for the total transaction made by each user')
df_customer = df.groupby(by = 'User_ID')['Purchase'].sum()
sns.histplot(data = df_customer, kde = True, bins = 200)
plt.plot()
```

[326]: []



```
[327]: plt.figure(figsize = (15, 6))
plt.subplot(1, 2, 1)
plt.title('Distribution of purchase per transaction for males')
df_male = df[df['Gender'] == 'M']
sns.histplot(data = df_male, x = 'Purchase')
plt.yticks(np.arange(0, 22550, 2500))
plt.subplot(1, 2, 2)
plt.title('Distribution of purchase per transaction for females')
df_female = df[df['Gender'] == 'F']
sns.histplot(data = df_female, x = 'Purchase')
plt.yticks(np.arange(0, 22550, 2500))
plt.show()
```



```
[328]: df_cust_gender = pd.DataFrame(df.groupby(by = ['Gender', 'User_ID'])['Purchase'].sum()).reset_index().rename(columns = {'Purchase' : 'Total_Purchase'})
df_cust_gender
```

```
[328]:
```

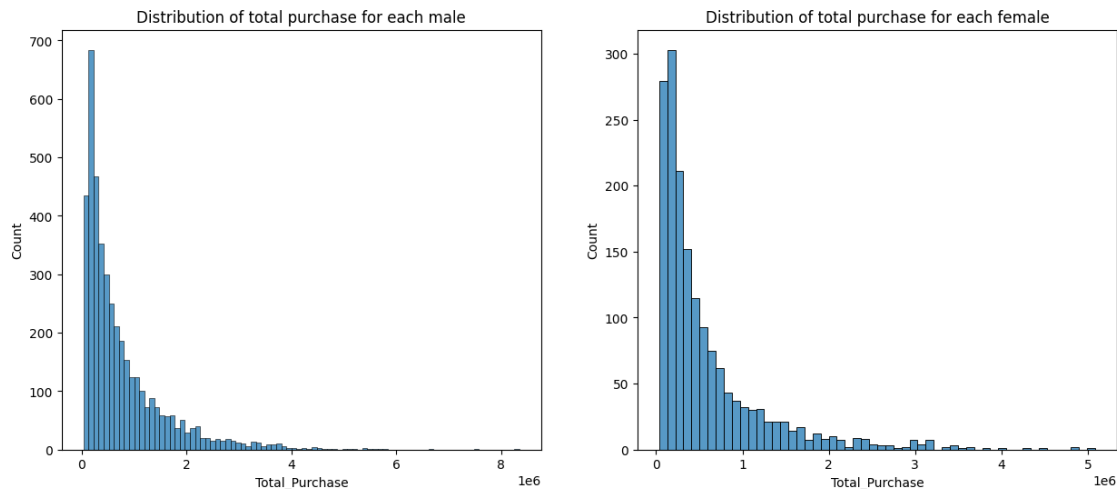
|      | Gender | User_ID | Total_Purchase |
|------|--------|---------|----------------|
| 0    | F      | 1000001 | 306287         |
| 1    | F      | 1000006 | 335275         |
| 2    | F      | 1000010 | 1832800        |
| 3    | F      | 1000011 | 496816         |
| 4    | F      | 1000016 | 142589         |
| ...  | ...    | ...     | ...            |
| 5886 | M      | 1006030 | 643256         |
| 5887 | M      | 1006032 | 410662         |
| 5888 | M      | 1006033 | 408143         |
| 5889 | M      | 1006034 | 132041         |

5890            M   1006040            1344711

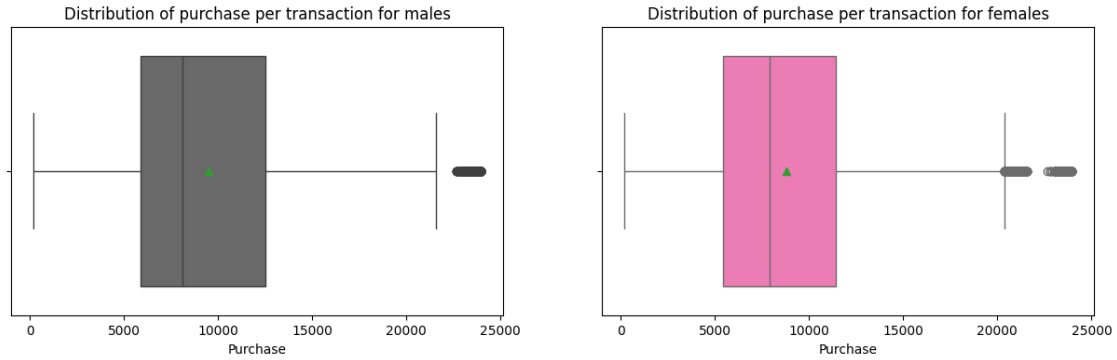
[5891 rows x 3 columns]

```
[329]: df_male_customer = df_cust_gender.loc[df_cust_gender['Gender'] == 'M']
df_female_customer = df_cust_gender.loc[df_cust_gender['Gender'] == 'F']
```

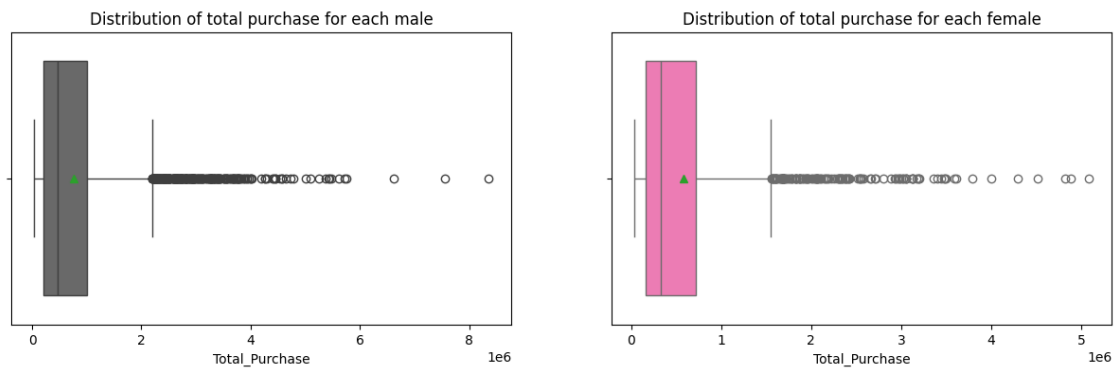
```
[330]: plt.figure(figsize = (15, 6))
plt.subplot(1, 2, 1)
plt.title('Distribution of total purchase for each male')
sns.histplot(data = df_male_customer, x = 'Total_Purchase')
plt.subplot(1, 2, 2)
plt.title('Distribution of total purchase for each female')
df_female = df[df['Gender'] == 'F']
sns.histplot(data = df_female_customer, x = 'Total_Purchase')
plt.show()
```



```
[331]: plt.figure(figsize = (15, 4))
plt.subplot(1, 2, 1)
plt.title('Distribution of purchase per transaction for males')
sns.boxplot(data = df_male, x = 'Purchase', showmeans = True, color = 'dimgray')
plt.subplot(1, 2, 2)
plt.title('Distribution of purchase per transaction for females')
sns.boxplot(data = df_female, x = 'Purchase', showmeans = True, color = 'hotpink')
plt.show()
```



```
[332]: plt.figure(figsize = (15, 4))
plt.subplot(1, 2, 1)
plt.title('Distribution of total purchase for each male')
sns.boxplot(data = df_male_customer, x = 'Total_Purchase', showmeans = True,
            color = 'dimgray')
plt.subplot(1, 2, 2)
plt.title('Distribution of total purchase for each female')
sns.boxplot(data = df_female_customer, x = 'Total_Purchase', showmeans = True,
            color = 'hotpink')
plt.show()
```



```
[333]: df['Age'].unique()
```

```
[333]: ['0-17', '55+', '26-35', '46-50', '51-55', '36-45', '18-25']
Categories (7, object): ['0-17', '18-25', '26-35', '36-45', '46-50', '51-55', '55+']
```

```
[334]: plt.figure(figsize = (12, 6))
plt.title('Distribution of number of Transactions Made based on Age',
        fontsize = 20,
```

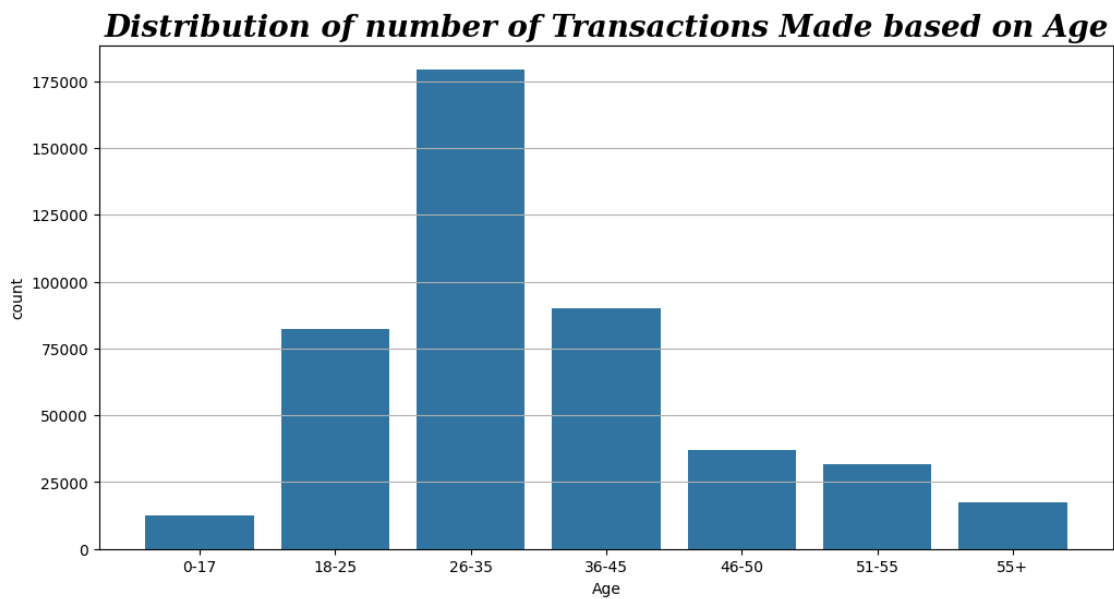


```

        fontweight = 600,
        fontstyle = 'oblique',
        fontfamily = 'serif')
plt.yticks(np.arange(0, 250001, 25000))
plt.grid('y')
sns.countplot(data = df, x = 'Age',
              order = ['0-17', '18-25', '26-35', '36-45', '46-50', '51-55',
↪ '55+'])
plt.plot()

```

[334]: []

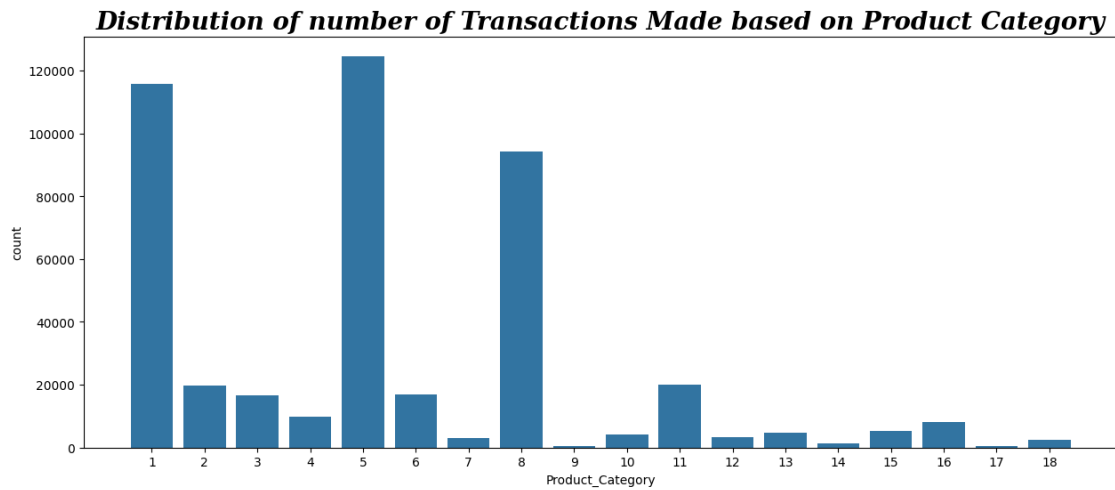


```

[335]: plt.figure(figsize = (15, 6))
plt.title('Distribution of number of Transactions Made based on Product_
↪ Category', fontsize = 20, fontweight = 600, fontstyle = 'oblique',
↪ fontfamily = 'serif')
sns.countplot(data = df, x = 'Product_Category')
plt.plot()

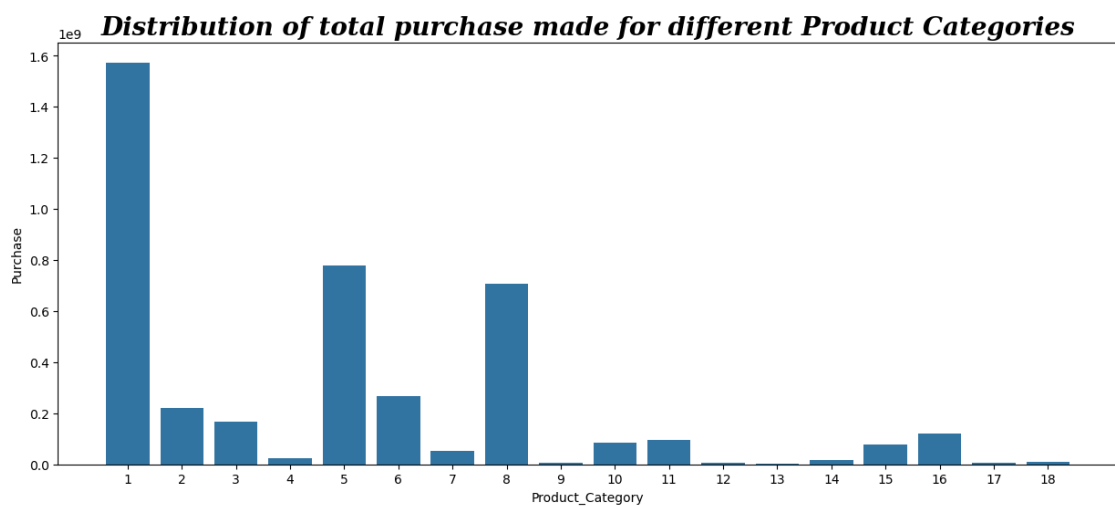
```

[335]: []



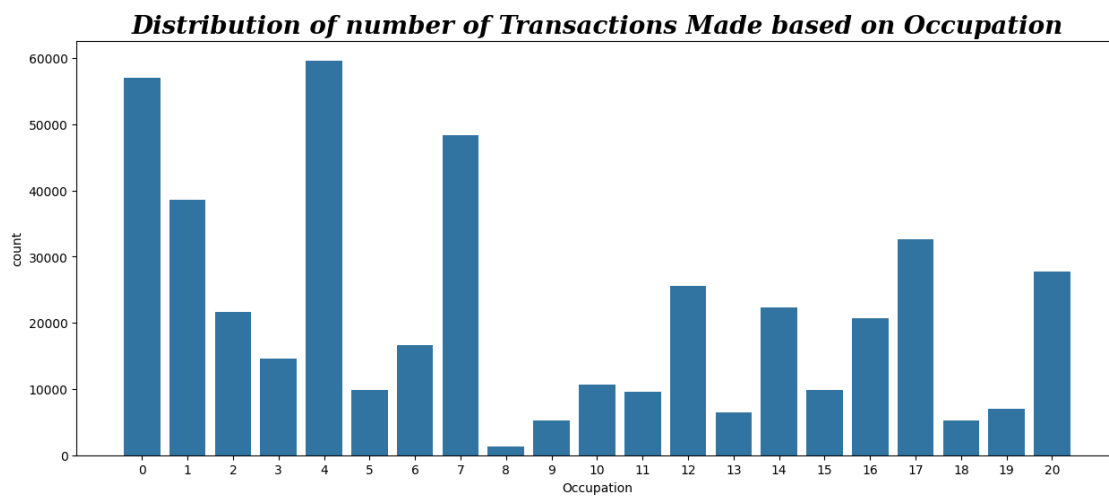
```
[336]: df_product_category = df.groupby(by = 'Product_Category')['Purchase'].sum().
        ↪to_frame().reset_index()
plt.figure(figsize = (15, 6))
plt.title('Distribution of total purchase made for different Product_
        ↪Categories',
          fontsize = 20,
          fontweight = 600,
          fontstyle = 'oblique',
          fontfamily = 'serif')
sns.barplot(data = df_product_category, x = 'Product_Category', y = 'Purchase')
plt.plot()
```

[336]: []



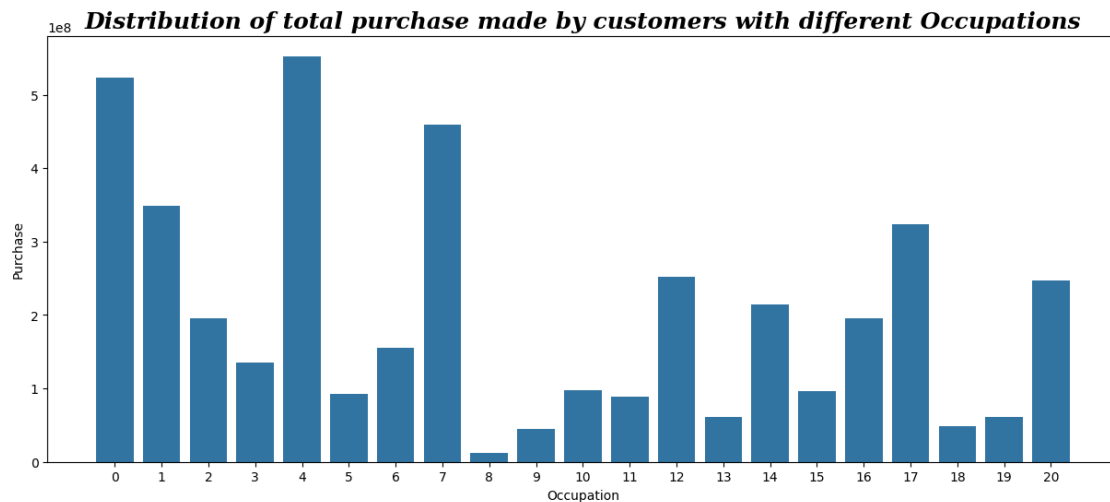
```
[337]: plt.figure(figsize = (15, 6))
plt.title('Distribution of number of Transactions Made based on Occupation',
          fontsize = 20,
          fontweight = 600,
          fontstyle = 'oblique',
          fontfamily = 'serif')
sns.countplot(data = df, x = 'Occupation')
plt.plot()
```

[337]: []



```
[338]: df_occupation = df.groupby(by = 'Occupation')['Purchase'].sum().to_frame().
        ↪reset_index()
plt.figure(figsize = (15, 6))
plt.title('Distribution of total purchase made by customers with different_
        ↪Occupations',
          fontsize = 18,
          fontweight = 600,
          fontstyle = 'oblique',
          fontfamily = 'serif')
sns.barplot(data = df_occupation, x = 'Occupation', y = 'Purchase')
plt.plot()
```

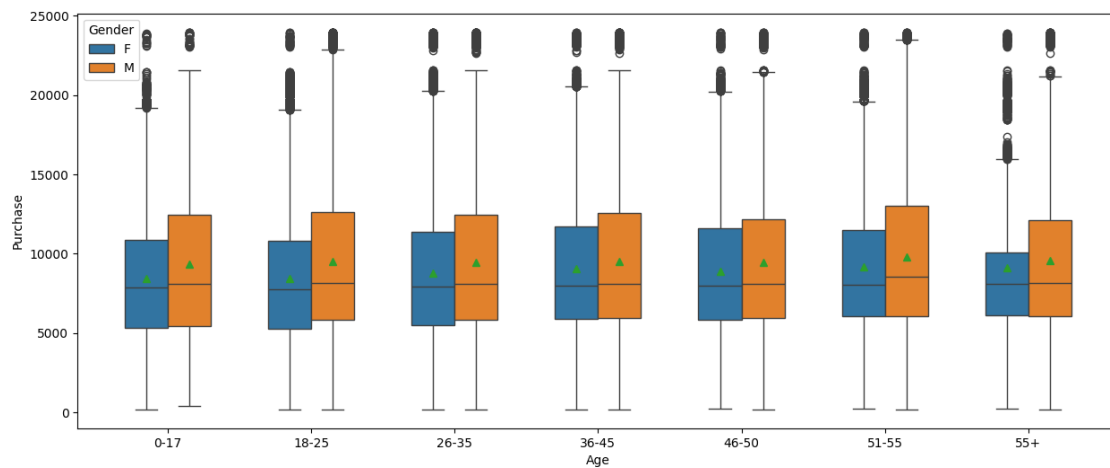
[338]: []



## Bivariate Analysis

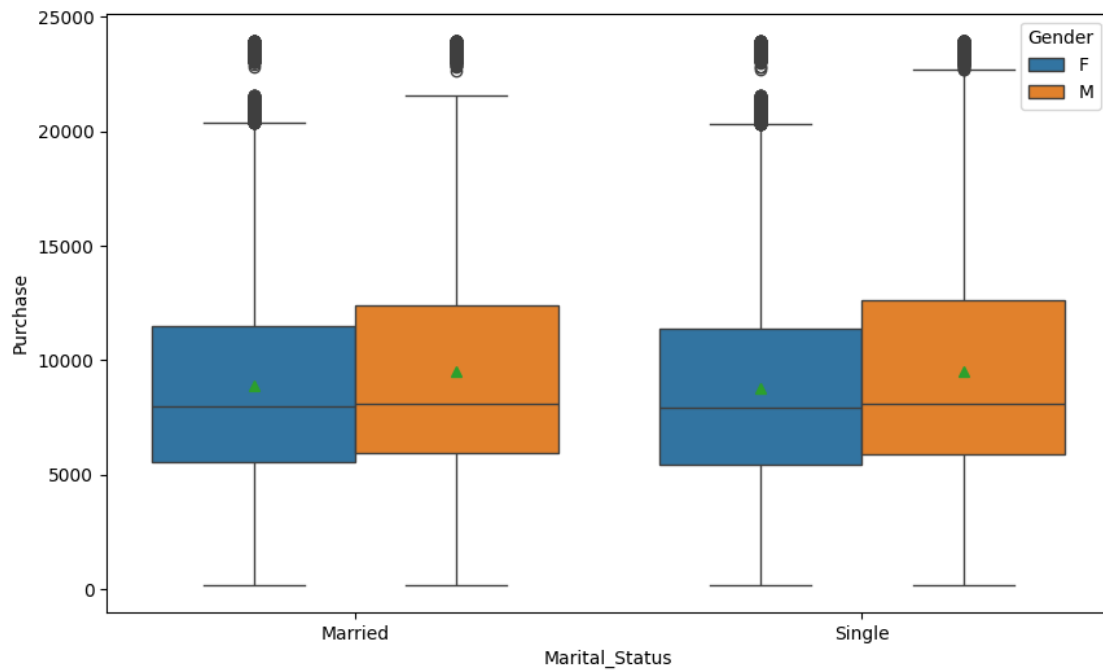
```
[339]: plt.figure(figsize = (15, 6))
sns.boxplot(data = df, x = 'Age', y = 'Purchase', hue = 'Gender', showmeans = True, width = 0.6)
plt.plot()
```

[339]: []



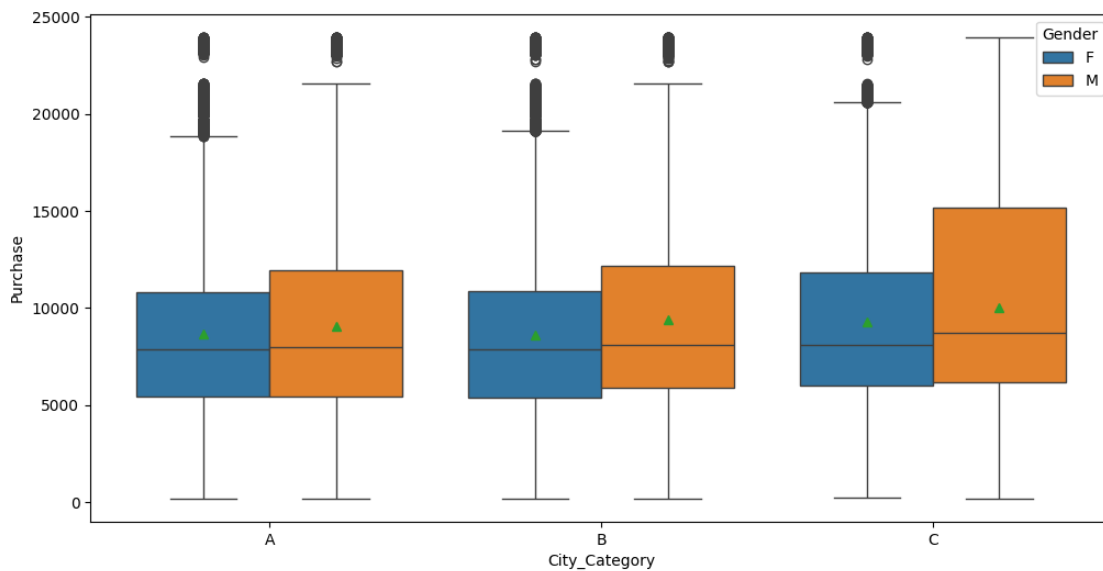
```
[340]: plt.figure(figsize = (10, 6))
sns.boxplot(data = df, x = 'Marital_Status', y = 'Purchase', hue = 'Gender', showmeans = True, width = 0.8)
plt.plot()
```

[340]: []



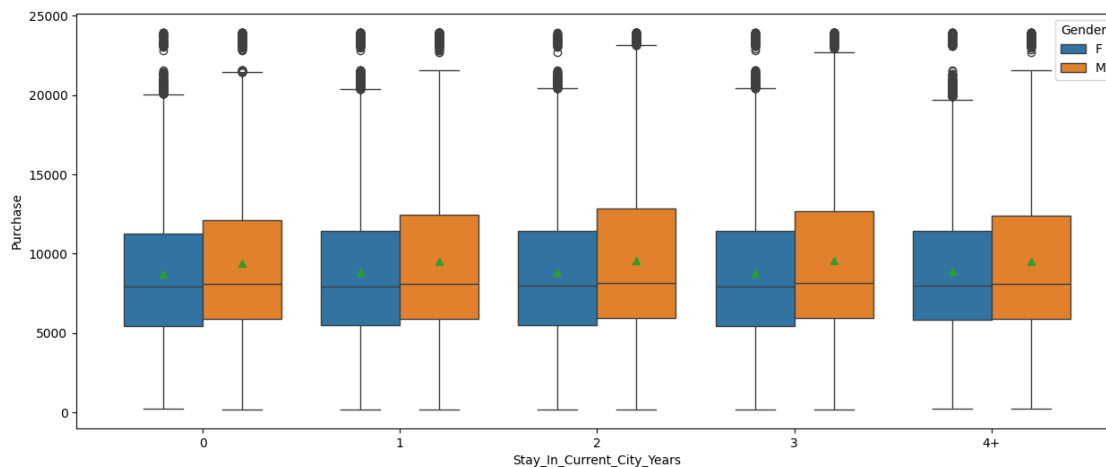
```
[341]: plt.figure(figsize = (12, 6))
sns.boxplot(data = df, x = 'City_Category', y = 'Purchase', hue = 'Gender',
            showmeans = True)
plt.plot()
```

[341]: []



```
[342]: plt.figure(figsize = (15, 6))
sns.boxplot(data = df, x = 'Stay_In_Current_City_Years', y = 'Purchase', hue = 'Gender', showmeans = True)
plt.plot()
```

[342]: []



Determining the mean purchase made by each user

```
[343]: # How the deviations vary for different sample sizes ?
df_male_customer
```

```
[343]:
```

|      | Gender | User_ID | Total_Purchase |
|------|--------|---------|----------------|
| 1666 | M      | 1000002 | 678303         |
| 1667 | M      | 1000003 | 302383         |
| 1668 | M      | 1000004 | 167871         |
| 1669 | M      | 1000005 | 657922         |
| 1670 | M      | 1000007 | 199125         |
| ...  | ...    | ...     | ...            |
| 5886 | M      | 1006030 | 643256         |
| 5887 | M      | 1006032 | 410662         |
| 5888 | M      | 1006033 | 408143         |
| 5889 | M      | 1006034 | 132041         |
| 5890 | M      | 1006040 | 1344711        |

[4225 rows x 3 columns]

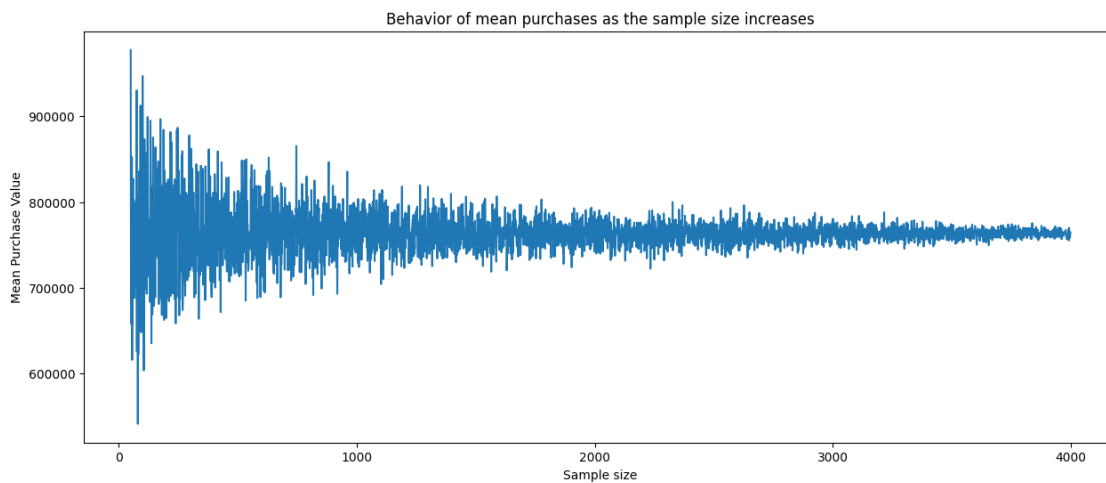
```
[344]: mean_purchases = []
for sample_size in range(50, 4000):
```

```
sample_mean = df_male_customer['Total_Purchase'].sample(sample_size).mean()
mean_purchases.append(sample_mean)
```

```
[345]: # Creating a plot using matplotlib to visualize the trend of the mean purchases
        # as the sample size increases
```

```
plt.figure(figsize = (15, 6))
plt.title('Behavior of mean purchases as the sample size increases')
plt.plot(np.arange(50, 4000), mean_purchases)
plt.xticks(np.arange(0, 10001, 1000))
plt.xlabel('Sample size')
plt.ylabel('Mean Purchase Value')
plt.plot()
```

```
[345]: []
```



```
[346]: # Finding the confidence interval of each male's total spending on the Black
        ↪ Friday
```

```
means_male = []
size = df_male_customer['Total_Purchase'].shape[0]
for bootstrapped_sample in range(10000):
    sample_mean = df_male_customer['Total_Purchase'].sample(size, replace =
    ↪ True).mean()
    means_male.append(sample_mean)
```

```
[347]: plt.figure(figsize = (15, 6))      # setting the figure size of the plot

sns.histplot(means_male, kde = True, bins = 100, fill = True, element = 'step')
```

```

# Above line plots a histogram of the data contained in the `means_male`
↳variable.

male_ll_90 = np.percentile(means_male, 5)
    # calculating the lower limit of the 90% confidence interval
male_ul_90 = np.percentile(means_male, 95)
    # calculating the upper limit of the 90% confidence interval
plt.axvline(male_ll_90, label = f'male_ll_90 : {round(male_ll_90, 2)}',
↳linestyle = '--')
    # adding a vertical line at the lower limit of the 90% confidence interval
plt.axvline(male_ul_90, label = f'male_ul_90 : {round(male_ul_90, 2)}',
↳linestyle = '--')

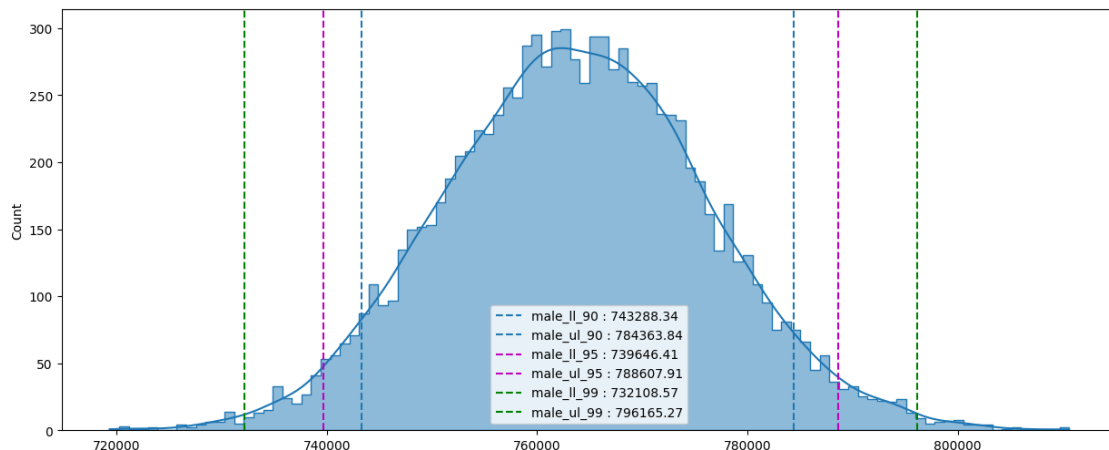
male_ll_95 = np.percentile(means_male, 2.5)
male_ul_95 = np.percentile(means_male, 97.5)
plt.axvline(male_ll_95, label = f'male_ll_95 : {round(male_ll_95, 2)}',
↳linestyle = '--', color = 'm')
plt.axvline(male_ul_95, label = f'male_ul_95 : {round(male_ul_95, 2)}',
↳linestyle = '--', color = 'm')

male_ll_99 = np.percentile(means_male, 0.5)
male_ul_99 = np.percentile(means_male, 99.5)
plt.axvline(male_ll_99, label = f'male_ll_99 : {round(male_ll_99, 2)}',
↳linestyle = '--', color = 'g')
plt.axvline(male_ul_99, label = f'male_ul_99 : {round(male_ul_99, 2)}',
↳linestyle = '--', color = 'g')

plt.legend()
plt.plot()

```

[347]: []





Through the bootstrapping method, we have been able to estimate the confidence interval for the total purchase made by each male customer on Black Friday at Walmart, despite having data for only 4225 male individuals. This provides us with a reasonable approximation of the range within which the total purchase of each male customer falls, with a certain level of confidence

```
[348]: print(f"The population mean of total spending of each male will be_
        ↪approximately = {np.round(np.mean(means_male), 2)} ")
```

The population mean of total spending of each male will be approximately = 763684.13

For Females

```
[349]: # How the deviations vary for different sample sizes ?
        df_female_customer
```

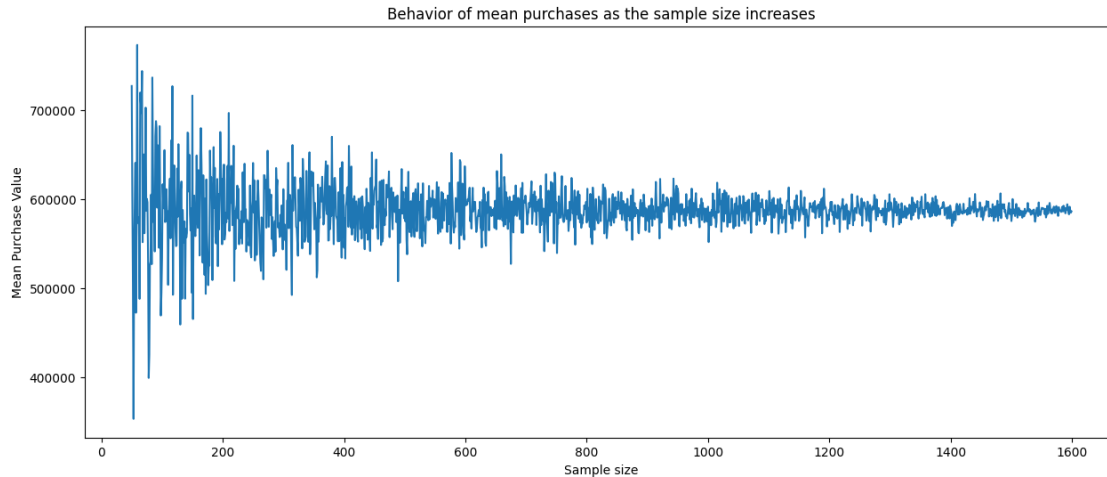
```
[349]:      Gender  User_ID  Total_Purchase
0         F  1000001         306287
1         F  1000006         335275
2         F  1000010        1832800
3         F  1000011         496816
4         F  1000016         142589
...
1661      F  1006035         783664
1662      F  1006036        3123961
1663      F  1006037         924666
1664      F  1006038          41479
1665      F  1006039        517795
```

[1666 rows x 3 columns]

```
[350]: mean_purchases = []
        for sample_size in range(50, 1600):
            sample_mean = df_female_customer['Total_Purchase'].sample(sample_size).
            ↪mean()
            mean_purchases.append(sample_mean)
```

```
[351]: plt.figure(figsize = (15, 6))
        plt.title('Behavior of mean purchases as the sample size increases')
        plt.plot(np.arange(50, 1600), mean_purchases)
        plt.xlabel('Sample size')
        plt.ylabel('Mean Purchase Value')
        plt.plot()
```

```
[351]: []
```



Finding the confidence interval of each female's total spending on the Black Friday

```
[352]: means_female = []
size = df_female_customer['Total_Purchase'].shape[0]
for bootstrapped_sample in range(10000):
    sample_mean = df_female_customer['Total_Purchase'].sample(size, replace =
↳ True).mean()
    means_female.append(sample_mean)
```

```
[353]: plt.figure(figsize = (15, 6))      # setting the figure size of the plot

sns.histplot(means_female, kde = True, bins = 100, fill = True, element =
↳ 'step')

female_ll_90 = np.percentile(means_female, 5)

female_ul_90 = np.percentile(means_female, 95)

plt.axvline(female_ll_90, label = f'female_ll_90 : {round(female_ll_90, 2)}',
↳ linestyle = '--')

plt.axvline(female_ul_90, label = f'female_ul_90 : {round(female_ul_90, 2)}',
↳ linestyle = '--')

female_ll_95 = np.percentile(means_female, 2.5)
female_ul_95 = np.percentile(means_female, 97.5)
plt.axvline(female_ll_95, label = f'female_ll_95 : {round(female_ll_95, 2)}',
↳ linestyle = '--', color = 'm')
plt.axvline(female_ul_95, label = f'female_ul_95 : {round(female_ul_95, 2)}',
↳ linestyle = '--', color = 'm')
```

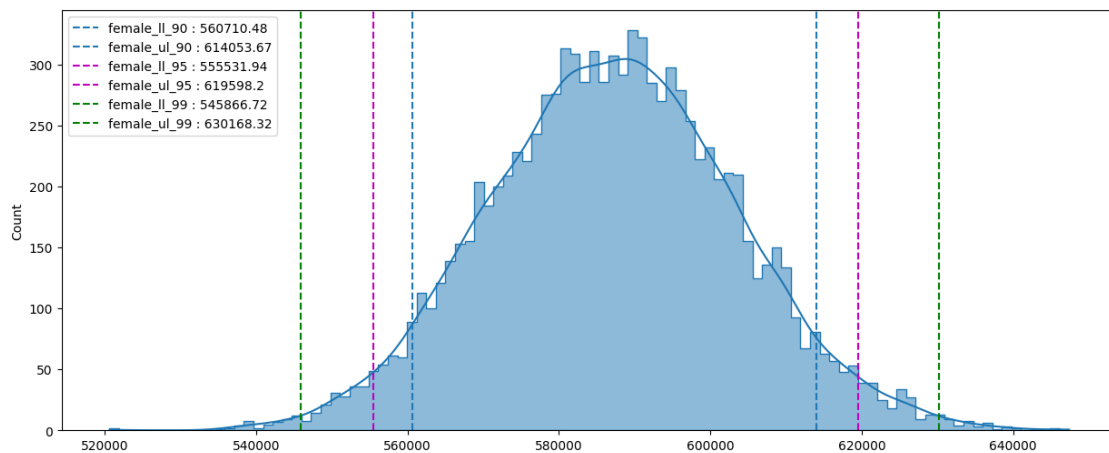
```

female_ll_99 = np.percentile(means_female, 0.5)
female_ul_99 = np.percentile(means_female, 99.5)
plt.axvline(female_ll_99, label = f'female_ll_99 : {round(female_ll_99, 2)}',
            linestyle = '--', color = 'g')
plt.axvline(female_ul_99, label = f'female_ul_99 : {round(female_ul_99, 2)}',
            linestyle = '--', color = 'g')

plt.legend()
plt.plot()

```

[353]: []



Through the bootstrapping method, we have been able to estimate the confidence interval for the total purchase made by each female customer on Black Friday at Walmart, despite having data for only 1666 female individuals. This provides us with a reasonable approximation of the range within which the total purchase of each female customer falls, with a certain level of confidence.

```

[354]: print(f"The population mean of total spending of each female will be
         approximately = {np.round(np.mean(means_female), 2)} ")

```

The population mean of total spending of each female will be approximately = 587113.38

Comparison of distributions of male's total purchase amount and female's total purchase amount

```

[355]: plt.figure(figsize = (18, 8))

# The first histogram represents the distribution of means_male with gray color
# having
# KDE (Kernel Density Estimation) curves enabled for smooth representation.

```

```

sns.histplot(means_male,
             kde = True,
             bins = 100,
             fill = True,
             element = 'step',
             color = 'gray',
             legend = True)

# Multiple vertical lines are plotted to represent the lower and upper limits
# for confidence intervals at different confidence levels
plt.axvline(male_ll_90, label = f'male_ll_90 : {round(male_ll_90, 2)}',
            linestyle = '--')
plt.axvline(male_ul_90, label = f'male_ul_90 : {round(male_ul_90, 2)}',
            linestyle = '--')
plt.axvline(male_ll_95, label = f'male_ll_95 : {round(male_ll_95, 2)}',
            linestyle = '--', color = 'm')
plt.axvline(male_ul_95, label = f'male_ul_95 : {round(male_ul_95, 2)}',
            linestyle = '--', color = 'm')
plt.axvline(male_ll_99, label = f'male_ll_99 : {round(male_ll_99, 2)}',
            linestyle = '--', color = 'g')
plt.axvline(male_ul_99, label = f'male_ul_99 : {round(male_ul_99, 2)}',
            linestyle = '--', color = 'g')

# The second histogram represents the distribution of means_female with magenta
color
# KDE (Kernel Density Estimation) curves enabled for smooth representation.
sns.histplot(means_female,
             kde = True,
             bins = 100,
             fill = True,
             element = 'step',
             color = 'magenta',
             legend = True)

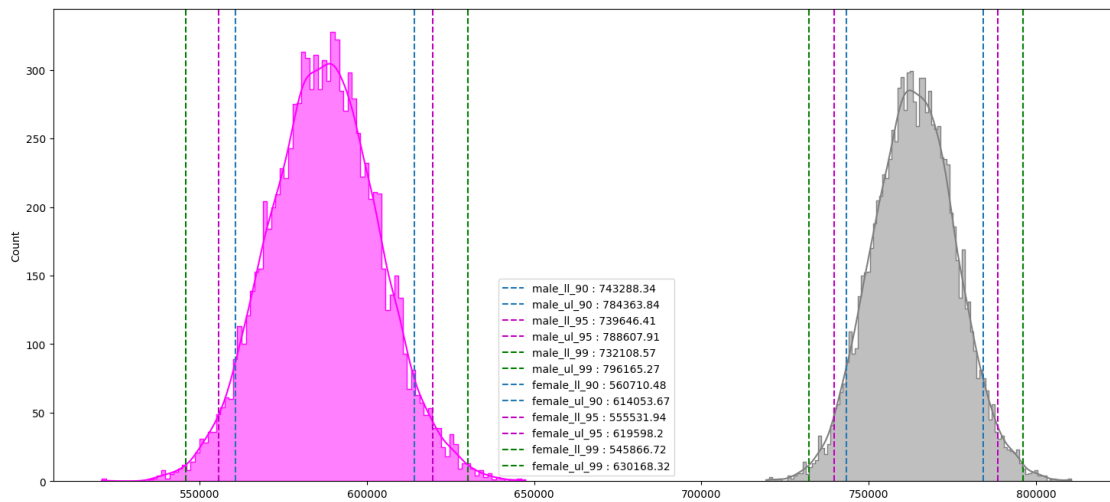
# Multiple vertical lines are plotted to represent the lower and upper limits
# for confidence intervals at different confidence levels
plt.axvline(female_ll_90, label = f'female_ll_90 : {round(female_ll_90, 2)}',
            linestyle = '--')
plt.axvline(female_ul_90, label = f'female_ul_90 : {round(female_ul_90, 2)}',
            linestyle = '--')
plt.axvline(female_ll_95, label = f'female_ll_95 : {round(female_ll_95, 2)}',
            linestyle = '--', color = 'm')
plt.axvline(female_ul_95, label = f'female_ul_95 : {round(female_ul_95, 2)}',
            linestyle = '--', color = 'm')
plt.axvline(female_ll_99, label = f'female_ll_99 : {round(female_ll_99, 2)}',
            linestyle = '--', color = 'g')

```

```
plt.axvline(female_ul_99, label = f'female_ul_99 : {round(female_ul_99, 2)}',
            linestyle = '--', color = 'g')

plt.legend()
plt.plot()
```

[355]: []



It can be clearly seen from the above chart that the distribution of males' total purchase amount lies well towards the right of females' total purchase amount. We can conclude that, on average, males tend to spend more on purchases compared to females. This observation suggests a potential difference in spending behavior between genders.

There could be several reasons why males are spending more than females:

**Product preferences:** Males may have a higher tendency to purchase products that are generally more expensive or fall into higher price categories. This could include items such as electronics, gadgets, or luxury goods.

**Income disparity:** There may be an income disparity between males and females, with males having higher earning potential or occupying higher-paying job roles. This can lead to a difference in purchasing power and ability to spend more on products.

**Consumption patterns:** Males might exhibit different consumption patterns, such as being more inclined towards hobbies or interests that require higher spending, such as sports equipment, gaming, or collectibles.

**Marketing and advertising targeting:** Advertisers and marketers may target males with products or services that are positioned at higher price points. This targeted marketing approach can influence purchasing decisions and contribute to males spending more.

It's important to note that these reasons are general observations and may not apply universally. Individual preferences, personal financial situations, and various other factors can also influence

spending patterns.

Determining the mean purchase made by each user belonging to different Marital Status

```
[356]: df_single = df.loc[df['Marital_Status'] == 'Single']
df_married = df.loc[df['Marital_Status'] == 'Married']
```

```
[357]: df_single = df_single.groupby('User_ID')['Purchase'].sum().to_frame().
        ↪reset_index().rename(columns = {'Purchase' : 'Total_Purchase'})
df_married = df_married.groupby('User_ID')['Purchase'].sum().to_frame().
        ↪reset_index().rename(columns = {'Purchase' : 'Total_Purchase'})
```

For Singles

```
[358]: df_single
```

```
[358]:
```

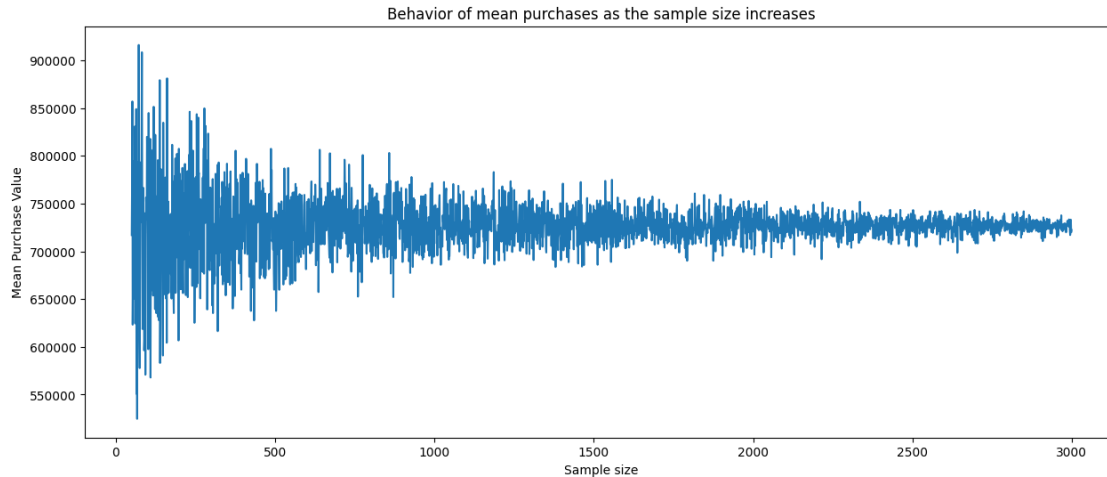
|      | User_ID | Total_Purchase |
|------|---------|----------------|
| 0    | 1000001 | 306287         |
| 1    | 1000002 | 678303         |
| 2    | 1000003 | 302383         |
| 3    | 1000006 | 335275         |
| 4    | 1000009 | 530527         |
| ...  | ...     | ...            |
| 3412 | 1006034 | 132041         |
| 3413 | 1006035 | 783664         |
| 3414 | 1006037 | 924666         |
| 3415 | 1006038 | 41479          |
| 3416 | 1006040 | 1344711        |

[3417 rows x 2 columns]

```
[359]: # How the deviations vary for different sample sizes ?
mean_purchases = []
for sample_size in range(50, 3000):
    sample_mean = df_single['Total_Purchase'].sample(sample_size).mean()
    mean_purchases.append(sample_mean)

plt.figure(figsize = (15, 6))
plt.title('Behavior of mean purchases as the sample size increases')
plt.plot(np.arange(50, 3000), mean_purchases)
plt.xlabel('Sample size')
plt.ylabel('Mean Purchase Value')
plt.plot()
```

```
[359]: []
```



```
[360]: # Finding the confidence interval of each single's total spending on the Black
        ↪Friday
single_means = []
size = df_single['Total_Purchase'].shape[0]
for bootstrapped_sample in range(10000):
    sample_mean = df_single['Total_Purchase'].sample(size, replace = True).
    ↪mean()
    single_means.append(sample_mean)
```

```
[361]: plt.figure(figsize = (15, 6))      # setting the figure size of the plot

sns.histplot(single_means, kde = True, bins = 100, fill = True, element =
    ↪'step')

# Above line plots a histogram of the data contained in the `single_means`
    ↪variable.
    # The `kde=True` argument adds a kernel density estimation line to the plot.
    # The `bins=100` argument sets the number of bins for the histogram

# Above line calculates the z-score corresponding to the 90% confidence level
    ↪using the
    # inverse of the cumulative distribution function (CDF) of a standard
    ↪normal distribution

single_ll_90 = np.percentile(single_means, 5)
    # calculating the lower limit of the 90% confidence interval
single_ul_90 = np.percentile(single_means, 95)
    # calculating the upper limit of the 90% confidence interval
```

```

plt.axvline(single_ll_90, label = f'single_ll_90 : {round(single_ll_90, 2)}',
            linestyle = '--')
    # adding a vertical line at the lower limit of the 90% confidence interval
plt.axvline(single_ul_90, label = f'single_ul_90 : {round(single_ul_90, 2)}',
            linestyle = '--')
    # adding a vertical line at the upper limit of the 90% confidence interval

# Similar steps are repeated for calculating and plotting the 95% and 99%
# confidence intervals,
    # with different line colors (`color='m` for 95% and `color='g` for 99%)

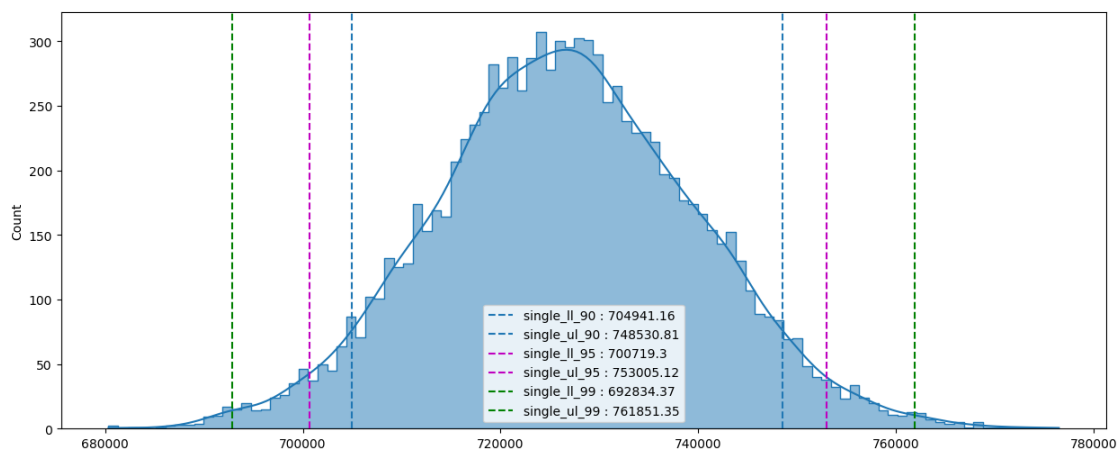
single_ll_95 = np.percentile(single_means, 2.5)
single_ul_95 = np.percentile(single_means, 97.5)
plt.axvline(single_ll_95, label = f'single_ll_95 : {round(single_ll_95, 2)}',
            linestyle = '--', color = 'm')
plt.axvline(single_ul_95, label = f'single_ul_95 : {round(single_ul_95, 2)}',
            linestyle = '--', color = 'm')

single_ll_99 = np.percentile(single_means, 0.5)
single_ul_99 = np.percentile(single_means, 99.5)
plt.axvline(single_ll_99, label = f'single_ll_99 : {round(single_ll_99, 2)}',
            linestyle = '--', color = 'g')
plt.axvline(single_ul_99, label = f'single_ul_99 : {round(single_ul_99, 2)}',
            linestyle = '--', color = 'g')

plt.legend()
plt.plot()

```

[361]: []





Through the bootstrapping method, we have been able to estimate the confidence interval for the total purchase made by each single customer on Black Friday at Walmart, despite having data for only 3417 individuals having single as marital status. This provides us with a reasonable approximation of the range within which the total purchase of each single customer falls, with a certain level of confidence.

```
[362]: print(f"The population mean of total spending of each single will be_
        ↳approximately = {np.round(np.mean(single_means), 2)} ")
```

The population mean of total spending of each single will be approximately =  
726598.14

For Marrieds

```
[363]: df_married
```

```
[363]:
```

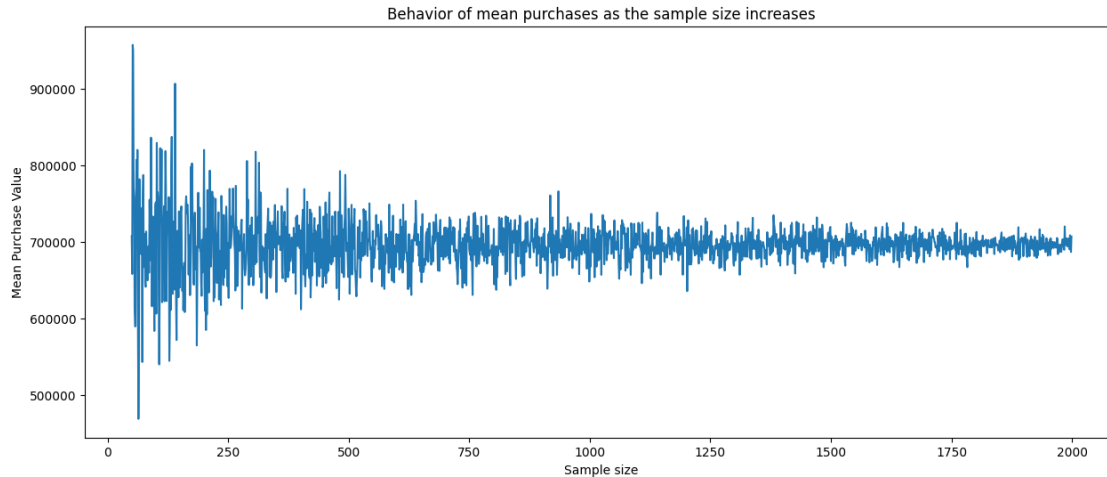
|      | User_ID | Total_Purchase |
|------|---------|----------------|
| 0    | 1000004 | 167871         |
| 1    | 1000005 | 657922         |
| 2    | 1000007 | 199125         |
| 3    | 1000008 | 727172         |
| 4    | 1000010 | 1832800        |
| ...  | ...     | ...            |
| 2469 | 1006029 | 102853         |
| 2470 | 1006030 | 643256         |
| 2471 | 1006033 | 408143         |
| 2472 | 1006036 | 3123961        |
| 2473 | 1006039 | 517795         |

[2474 rows x 2 columns]

```
[364]: # How the deviations vary for different sample sizes ?
mean_purchases = []
for sample_size in range(50, 2000):
    sample_mean = df_married['Total_Purchase'].sample(sample_size).mean()
    mean_purchases.append(sample_mean)

plt.figure(figsize = (15, 6))
plt.title('Behavior of mean purchases as the sample size increases')
plt.plot(np.arange(50, 2000), mean_purchases)
plt.xlabel('Sample size')
plt.ylabel('Mean Purchase Value')
plt.plot()
```

```
[364]: []
```



```
[365]: # Finding the confidence interval of each married's total spending on the Black
        ↪ Friday
```

```
married_means = []
size = df_married['Total_Purchase'].shape[0]
for bootstrapped_sample in range(10000):
    sample_mean = df_married['Total_Purchase'].sample(size, replace = True).
    ↪mean()
    married_means.append(sample_mean)
```

```
[366]: plt.figure(figsize = (15, 6))      # setting the figure size of the plot

sns.histplot(married_means, kde = True, bins = 100, fill = True, element =
    ↪'step')

# Above line plots a histogram of the data contained in the `married_means`
    ↪variable.
    # The `kde=True` argument adds a kernel density estimation line to the plot.
    # The `bins=100` argument sets the number of bins for the histogram

# Above line calculates the z-score corresponding to the 90% confidence level
    ↪using the
    # inverse of the cumulative distribution function (CDF) of a standard
    ↪normal distribution

married_ll_90 = np.percentile(married_means, 5)
    # calculating the lower limit of the 90% confidence interval
married_ul_90 = np.percentile(married_means, 95)
    # calculating the upper limit of the 90% confidence interval
```

```

plt.axvline(married_ll_90, label = f'married_ll_90 : {round(married_ll_90,
↪2)}', linestyle = '--')
    # adding a vertical line at the lower limit of the 90% confidence interval
plt.axvline(married_ul_90, label = f'married_ul_90 : {round(married_ul_90,
↪2)}', linestyle = '--')
    # adding a vertical line at the upper limit of the 90% confidence interval

# Similar steps are repeated for calculating and plotting the 95% and 99%
↪confidence intervals,
    # with different line colors (`color='m'` for 95% and `color='g'` for 99%)

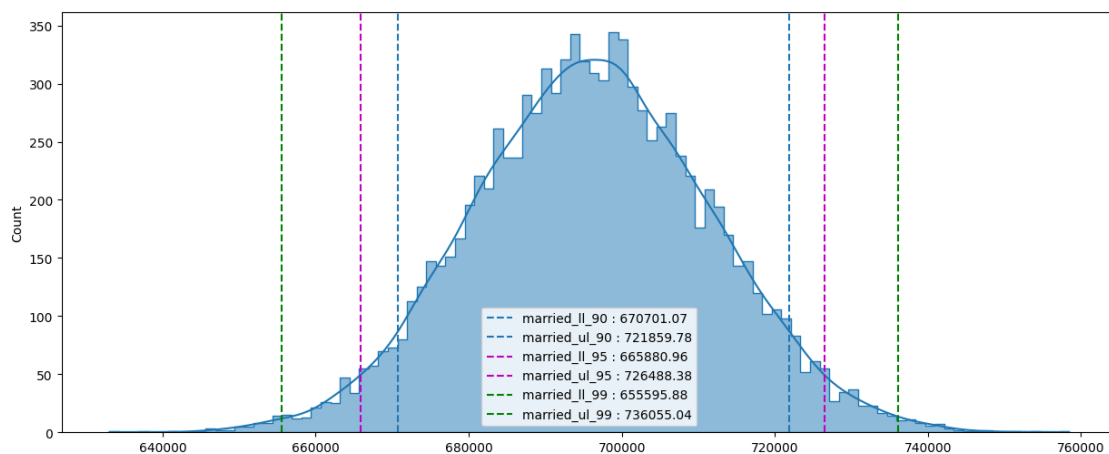
married_ll_95 = np.percentile(married_means, 2.5)
married_ul_95 = np.percentile(married_means, 97.5)
plt.axvline(married_ll_95, label = f'married_ll_95 : {round(married_ll_95,
↪2)}', linestyle = '--', color = 'm')
plt.axvline(married_ul_95, label = f'married_ul_95 : {round(married_ul_95,
↪2)}', linestyle = '--', color = 'm')

married_ll_99 = np.percentile(married_means, 0.5)
married_ul_99 = np.percentile(married_means, 99.5)
plt.axvline(married_ll_99, label = f'married_ll_99 : {round(married_ll_99,
↪2)}', linestyle = '--', color = 'g')
plt.axvline(married_ul_99, label = f'married_ul_99 : {round(married_ul_99,
↪2)}', linestyle = '--', color = 'g')

plt.legend()
plt.plot()

```

[366]: []



Through the bootstrapping method, we have been able to estimate the confidence interval for the total purchase made by each married customer on Black Friday at Walmart, despite having data for only 2474 individuals having married as marital status. This provides us with a reasonable approximation of the range within which the total purchase of each married customer falls, with a certain level of confidence.

```
[367]: print(f"The population mean of total spending of each male will be approximately = {np.round(np.mean(married_means), 2)} ")
```

The population mean of total spending of each male will be approximately = 696102.55

```
[368]: # Comparison of distributions of single's total purchase amount and married's total purchase amount
plt.figure(figsize = (18, 8))

sns.histplot(single_means,
              kde = True,
              bins = 100,
              fill = True,
              element = 'step',
              color = 'gray',
              legend = True)

plt.axvline(single_ll_90, label = f'single_ll_90 : {round(single_ll_90, 2)}',
             linestyle = '--')
plt.axvline(single_ul_90, label = f'single_ul_90 : {round(single_ul_90, 2)}',
             linestyle = '--')
plt.axvline(single_ll_95, label = f'single_ll_95 : {round(single_ll_95, 2)}',
             linestyle = '--', color = 'm')
plt.axvline(single_ul_95, label = f'single_ul_95 : {round(single_ul_95, 2)}',
             linestyle = '--', color = 'm')
plt.axvline(single_ll_99, label = f'single_ll_99 : {round(single_ll_99, 2)}',
             linestyle = '--', color = 'g')
plt.axvline(single_ul_99, label = f'single_ul_99 : {round(single_ul_99, 2)}',
             linestyle = '--', color = 'g')

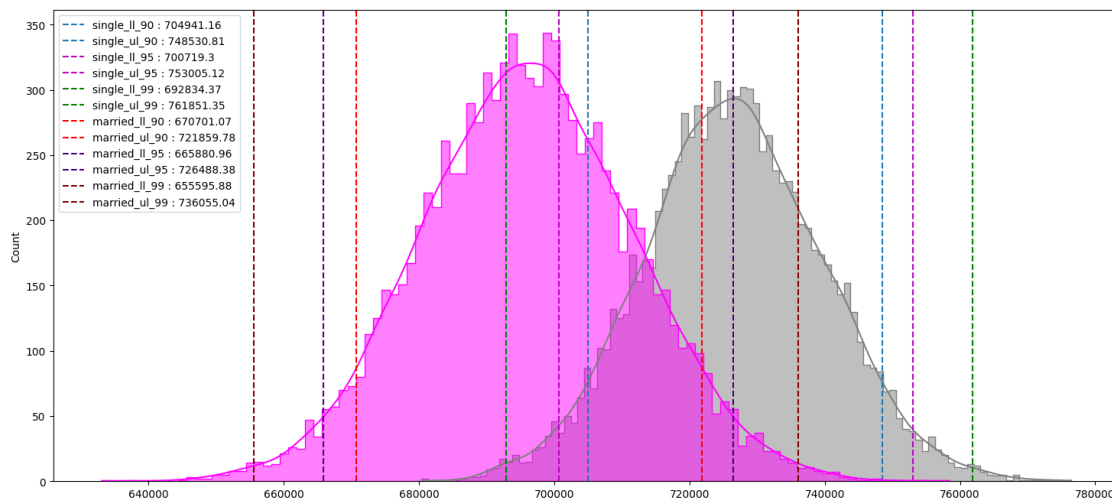
sns.histplot(married_means,
              kde = True,
              bins = 100,
              fill = True,
              element = 'step',
              color = 'magenta',
              legend = True)

plt.axvline(married_ll_90, label = f'married_ll_90 : {round(married_ll_90, 2)}',
             linestyle = '--', color = 'r')
```

```
plt.axvline(married_ul_90, label = f'married_ul_90 : {round(married_ul_90, 2)}', linestyle = '--', color = 'r')
plt.axvline(married_ll_95, label = f'married_ll_95 : {round(married_ll_95, 2)}', linestyle = '--', color = 'indigo')
plt.axvline(married_ul_95, label = f'married_ul_95 : {round(married_ul_95, 2)}', linestyle = '--', color = 'indigo')
plt.axvline(married_ll_99, label = f'married_ll_99 : {round(married_ll_99, 2)}', linestyle = '--', color = 'maroon')
plt.axvline(married_ul_99, label = f'married_ul_99 : {round(married_ul_99, 2)}', linestyle = '--', color = 'maroon')

plt.legend()
plt.plot()
```

[368]: []



It can be inferred from the above chart that the distributions of singles' total spending and married individuals' total spending overlap. It suggests that there is no significant difference in spending habits between these two groups. Here are some possible inferences that can be drawn from this:

**Relationship status does not strongly influence spending:** Being single or married does not appear to have a substantial impact on individuals' spending patterns. Other factors such as income, personal preferences, and financial priorities may play a more significant role in determining spending habits.

**Similar consumption patterns:** Singles and married individuals may have similar lifestyles and consumption patterns, leading to comparable spending behaviors. They may allocate their income in comparable ways, making similar purchasing decisions and spending on similar categories of products or services.

**Financial considerations:** Both singles and married individuals may have similar financial responsibilities and constraints, leading to similar spending levels. They may have similar obligations such

as housing costs, bills, and other financial commitments, which influence their overall spending capacity.

Individual differences outweigh relationship status: Other individual characteristics, such as personal values, interests, and financial habits, may have a more significant impact on spending behavior than relationship status. These factors can vary widely within each group, resulting in overlapping spending distributions.

Determining the mean purchase made by each user based on their age groups :

```
[369]: df['Age'].unique()
```

```
[369]: ['0-17', '55+', '26-35', '46-50', '51-55', '36-45', '18-25']
Categories (7, object): ['0-17', '18-25', '26-35', '36-45', '46-50', '51-55', '55+']
```

```
[370]: df_age_0_to_17 = df.loc[df['Age'] == '0-17']
df_age_18_to_25 = df.loc[df['Age'] == '18-25']
df_age_26_to_35 = df.loc[df['Age'] == '26-35']
df_age_36_to_45 = df.loc[df['Age'] == '36-45']
df_age_46_to_50 = df.loc[df['Age'] == '46-50']
df_age_51_to_55 = df.loc[df['Age'] == '51-55']
df_age_above_55 = df.loc[df['Age'] == '55+']
```

```
[371]: df_age_0_to_17 = df_age_0_to_17.groupby(by = 'User_ID')['Purchase'].sum().
↳to_frame().reset_index().rename(columns = {'Purchase' : 'Total_Purchase'})
df_age_18_to_25 = df_age_18_to_25.groupby(by = 'User_ID')['Purchase'].sum().
↳to_frame().reset_index().rename(columns = {'Purchase' : 'Total_Purchase'})
df_age_26_to_35 = df_age_26_to_35.groupby(by = 'User_ID')['Purchase'].sum().
↳to_frame().reset_index().rename(columns = {'Purchase' : 'Total_Purchase'})
df_age_36_to_45 = df_age_36_to_45.groupby(by = 'User_ID')['Purchase'].sum().
↳to_frame().reset_index().rename(columns = {'Purchase' : 'Total_Purchase'})
df_age_46_to_50 = df_age_46_to_50.groupby(by = 'User_ID')['Purchase'].sum().
↳to_frame().reset_index().rename(columns = {'Purchase' : 'Total_Purchase'})
df_age_51_to_55 = df_age_51_to_55.groupby(by = 'User_ID')['Purchase'].sum().
↳to_frame().reset_index().rename(columns = {'Purchase' : 'Total_Purchase'})
df_age_above_55 = df_age_above_55.groupby(by = 'User_ID')['Purchase'].sum().
↳to_frame().reset_index().rename(columns = {'Purchase' : 'Total_Purchase'})
```

For Age Group 0 - 17 years

```
[372]: df_age_0_to_17
```

```
[372]:
```

|   | User_ID | Total_Purchase |
|---|---------|----------------|
| 0 | 1000001 | 306287         |
| 1 | 1000019 | 1300942        |
| 2 | 1000051 | 195521         |
| 3 | 1000075 | 850360         |

```

4      1000086      242763
..      ...      ...
213    1005844      404795
214    1005953      501639
215    1005973      235364
216    1005989      383568
217    1006006      391634

```

[218 rows x 2 columns]

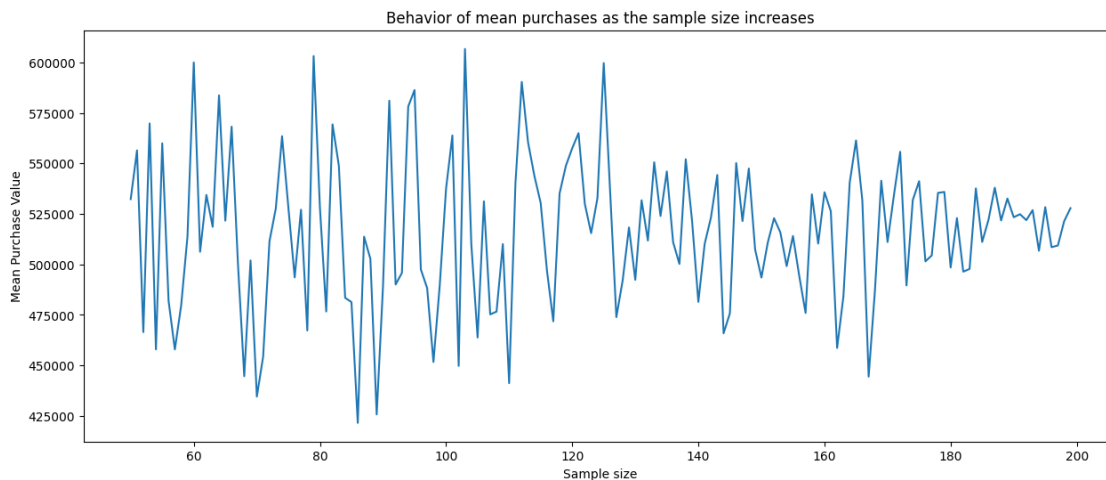
```

[373]: # How the deviations vary for different sample sizes ?
mean_purchases = []
for sample_size in range(50, 200):
    sample_mean = df_age_0_to_17['Total_Purchase'].sample(sample_size).mean()
    mean_purchases.append(sample_mean)

plt.figure(figsize = (15, 6))
plt.title('Behavior of mean purchases as the sample size increases')
plt.plot(np.arange(50, 200), mean_purchases)
plt.xlabel('Sample size')
plt.ylabel('Mean Purchase Value')
plt.plot()

```

[373]: []



```

[374]: # Finding the confidence interval of total spending for each individual in the
        ↳ age group 0 - 17 on the Black Friday
means = []
size = df_age_0_to_17['Total_Purchase'].shape[0]
for bootstrapped_sample in range(10000):

```

```

    sample_mean = df_age_0_to_17['Total_Purchase'].sample(size, replace = True).
↪mean()
    means.append(sample_mean)

```

```

[375]: plt.figure(figsize = (15, 6))      # setting the figure size of the plot

sns.histplot(means, kde = True, bins = 100, fill = True, element = 'step')

# Above line plots a histogram of the data contained in the `means` variable.
# The `kde=True` argument adds a kernel density estimation line to the plot.
# The `bins=100` argument sets the number of bins for the histogram

# Above line calculates the z-score corresponding to the 90% confidence level,
↪using the
# inverse of the cumulative distribution function (CDF) of a standard
↪normal distribution

ll_90 = np.percentile(means, 5)

ul_90 = np.percentile(means, 95)

plt.axvline(ll_90, label = f'll_90 : {round(ll_90, 2)}', linestyle = '--')

plt.axvline(ul_90, label = f'ul_90 : {round(ul_90, 2)}', linestyle = '--')

ll_95 = np.percentile(means, 2.5)
ul_95 = np.percentile(means, 97.5)
plt.axvline(ll_95, label = f'll_95 : {round(ll_95, 2)}', linestyle = '--',
↪color = 'm')
plt.axvline(ul_95, label = f'ul_95 : {round(ul_95, 2)}', linestyle = '--',
↪color = 'm')

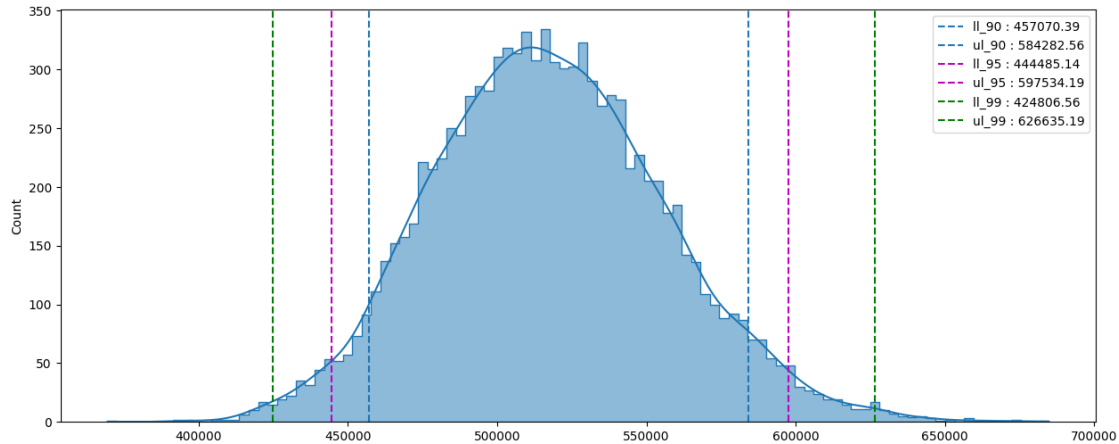
ll_99 = np.percentile(means, 0.5)
ul_99 = np.percentile(means, 99.5)
plt.axvline(ll_99, label = f'll_99 : {round(ll_99, 2)}', linestyle = '--',
↪color = 'g')
plt.axvline(ul_99, label = f'ul_99 : {round(ul_99, 2)}', linestyle = '--',
↪color = 'g')

plt.legend()
plt.plot()

```

[375]: []





Through the bootstrapping method, we have been able to estimate the confidence interval for the total purchase made by each individual in age group 0 - 17 years on Black Friday at Walmart, despite having data for only 218 individuals having age group 0 - 17 years. This provides us with a reasonable approximation of the range within which the total purchase of each individuals having age group 0 - 17 years falls, with a certain level of confidence.

```
[376]: print(f"The population mean of total spending of each customer in age group 0-17 will be approximately = {np.round(np.mean(means), 2)} ")
```

The population mean of total spending of each customer in age group 0 -17 will be approximately = 517433.26

For Age Group 18 - 25 years

```
[377]: df_age_18_to_25
```

```
[377]:
```

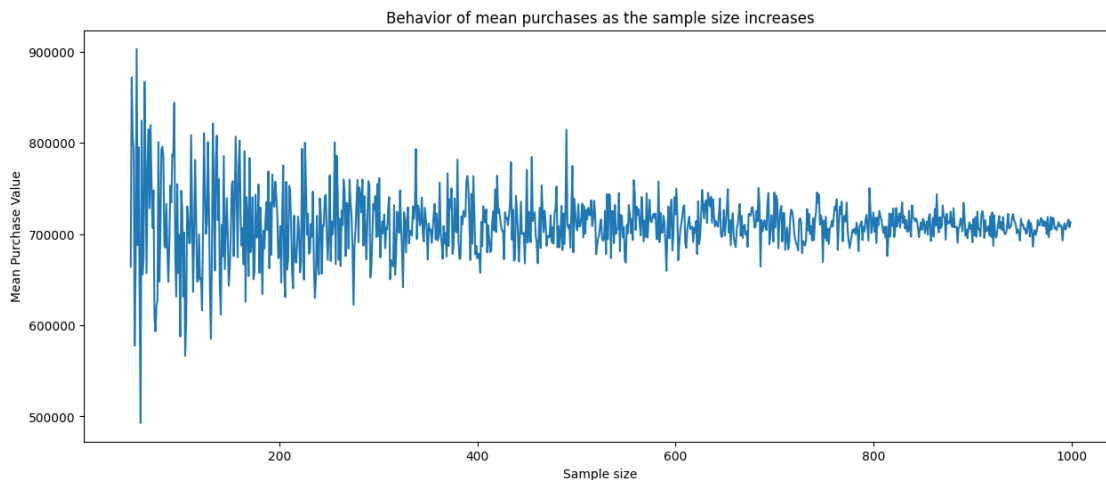
|      | User_ID | Total_Purchase |
|------|---------|----------------|
| 0    | 1000018 | 1744405        |
| 1    | 1000021 | 107069         |
| 2    | 1000022 | 1058973        |
| 3    | 1000025 | 494325         |
| 4    | 1000034 | 662183         |
| ...  | ...     | ...            |
| 1064 | 1005998 | 586332         |
| 1065 | 1006008 | 259682         |
| 1066 | 1006027 | 229793         |
| 1067 | 1006028 | 301007         |
| 1068 | 1006031 | 200624         |

[1069 rows x 2 columns]

```
[378]: # How the deviations vary for different sample sizes ?
mean_purchases = []
for sample_size in range(50, 1000):
    sample_mean = df_age_18_to_25['Total_Purchase'].sample(sample_size).mean()
    mean_purchases.append(sample_mean)

plt.figure(figsize = (15, 6))
plt.title('Behavior of mean purchases as the sample size increases')
plt.plot(np.arange(50, 1000), mean_purchases)
plt.xlabel('Sample size')
plt.ylabel('Mean Purchase Value')
plt.plot()
```

[378]: []



```
[379]: # Finding the confidence interval of total spending for each individual in the
        ↪ age group 18 - 25 on the Black Friday
means = []
size = df_age_18_to_25['Total_Purchase'].shape[0]
for bootstrapped_sample in range(10000):
    sample_mean = df_age_18_to_25['Total_Purchase'].sample(size, replace =
        ↪ True).mean()
    means.append(sample_mean)
```

```
[380]: plt.figure(figsize = (15, 6))      # setting the figure size of the plot

sns.histplot(means, kde = True, bins = 100, fill = True, element = 'step')

# Above line plots a histogram of the data contained in the `means` variable.
# The `kde=True` argument adds a kernel density estimation line to the plot.
```

```

# The `bins=100` argument sets the number of bins for the histogram

# Above line calculates the z-score corresponding to the 90% confidence level,
↳using the
    # inverse of the cumulative distribution function (CDF) of a standard
↳normal distribution

ll_90 = np.percentile(means, 5)

ul_90 = np.percentile(means, 95)

plt.axvline(ll_90, label = f'll_90 : {round(ll_90, 2)}', linestyle = '--')

plt.axvline(ul_90, label = f'ul_90 : {round(ul_90, 2)}', linestyle = '--')

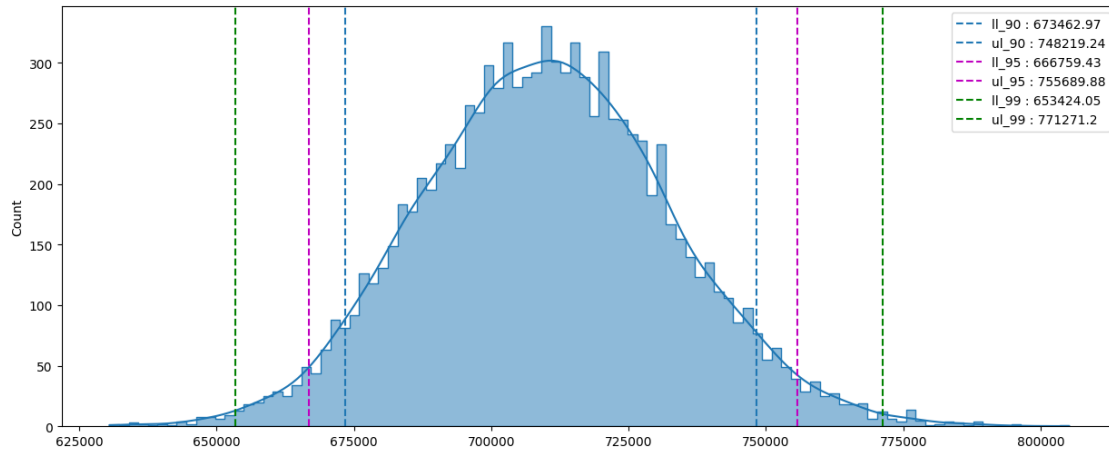
ll_95 = np.percentile(means, 2.5)
ul_95 = np.percentile(means, 97.5)
plt.axvline(ll_95, label = f'll_95 : {round(ll_95, 2)}', linestyle = '--',
↳color = 'm')
plt.axvline(ul_95, label = f'ul_95 : {round(ul_95, 2)}', linestyle = '--',
↳color = 'm')

ll_99 = np.percentile(means, 0.5)
ul_99 = np.percentile(means, 99.5)
plt.axvline(ll_99, label = f'll_99 : {round(ll_99, 2)}', linestyle = '--',
↳color = 'g')
plt.axvline(ul_99, label = f'ul_99 : {round(ul_99, 2)}', linestyle = '--',
↳color = 'g')

plt.legend()
plt.plot()

```

[380]: []



### Actionable insights

Out of every four transactions made on Black Friday in the Walmart stores, three are made by the males and one is made by the females.

82.33 % of the total transactions are made by the customers belonging to 11 occupations. These are 4, 0, 7, 1, 17, 20, 12, 14, 2, 16, 6 (Ordered in descending order of the total transactions' share.)

Majority of the transactions (53.75 % of total transactions) are made by the customers having 1 or 2 years of stay in the current city. 82.43% of the total transactions are made for only 5 Product Categories. These are, 5, 1, 8, 11 and 2.

There are 1666 unique female customers and 4225 unique male customers. Average number of transactions made by each Male on Black Friday is 98 while for Female it is 82.

On an average each male makes a total purchase of 925438.92 on Black Friday while for each female the figure is 712269.56.

76.72 % of the total revenue is generated from males.

Out of 5891 unique customers, 42 % of them are Married and 58 % of them are Single.

Average number of transactions made by each user with marital status Married is 91 and for Single it is 95.

On an average each Married customer makes a total purchase of 843469.79 on Black Friday while for each Single customer the figure is 880526.31.

59.05 % of the total revenue is generated from the customers who are Single.

Majority of the transactions are made by the customers whose age is between 26 and 45 years.

About 81.82% of the total transactions are made by customers of age between 18 and 50 years.

81.82 % of total unique customers have age between 18 and 50 years. Out of all unique customers, 35.85 % belong to the age group of 26 - 35 years, 19.81 % belong to the age group of 36 - 45 years, 18.15 % belong to the age group of 18 - 25 years, 9.01 % belong to the age group of 46 - 50 years.

Walmart generated 86.21 % of total revenue from customers in range 18 to 50 years on Black Friday. 39.87 % of the total revenue is generated from the customers having age group of 26 - 35 years, 20.15 % is generated from 36 - 45 years, 17.93 % from 18 - 25 years, 8.26 % from 46 - 50 years.

Majority of the total unique customers belong to the city C. 82.26 % of the total unique customers belong to city C and B.

Walmart generated 41.52 % of the total revenue from the customers belonging to the city B, 32.65 % from city C and 25.83 % from city A on Black Friday.

Top 5 product categories from which Walmart made 84.36 % of total revenue on Black Friday are 1, 5, 8, 6 and 2.

The population mean of total spending of each male will be approximately = 925156.36.

The population mean of total spending of each female will be approximately = 711789.37

The population mean of total spending of each single will be approximately = 880356.19

The population mean of total spending of each male will be approximately = 843632.08

The population mean of total spending of each customer in age group 0 -17 will be approximately = 617797.25

The population mean of total spending of each customer in age group 18 - 25 will be approximately = 854676.31

The population mean of total spending of each customer in age group 26 - 35 will be approximately = 989120.36

The population mean of total spending of each customer in age group 36 - 45 will be approximately = 879434.88

The population mean of total spending of each customer in age group 46 - 50 will be approximately = 792671.74

### **3 Recommendations**

**Targeted marketing:** Since the majority of transactions are made by males, it would be beneficial to tailor marketing strategies to cater to their preferences and needs. This could include specific promotions, product offerings, or advertising campaigns designed to attract male customers.

**Focus on popular occupations:** Given that 82.33% of transactions come from customers in 11 specific occupations, it would be wise to focus marketing efforts on these occupations. Understanding the needs and preferences of individuals in these occupations can help in creating targeted marketing campaigns and customized offers.

**Engage with new residents:** As a significant portion of transactions (53.75%) come from customers who have recently moved to the current city, it presents an opportunity to engage with these new residents. Targeted marketing, welcoming offers, and incentives for newcomers can help capture their loyalty and increase their spending.

**Emphasize popular product categories:** Since 82.43% of transactions are concentrated in just five product categories, allocating resources and promotions towards these categories can maximize

sales potential. Highlighting these popular categories and offering attractive deals can encourage more purchases.

**Increase focus on single customers:** Given that 59.05% of total revenue is generated by single customers, dedicating efforts to cater to their needs and preferences can help drive more sales. Understanding their motivations and targeting them with personalized offers can enhance their shopping experience and loyalty.

**Optimize revenue from specific age groups:** Since a majority of transactions are made by customers between the ages of 26 and 45, it is important to focus marketing efforts on this demographic. Offering products and services that align with their interests and values can maximize revenue generation.

**Location-based marketing:** With a significant number of customers belonging to specific cities, tailoring marketing strategies to target these locations can lead to better results. Allocating resources, promotions, and events based on the customer concentration in each city can help drive sales.

**Emphasize top-selling product categories:** The top five product categories generate a substantial portion of total revenue. Investing in these categories, ensuring a wide range of options and competitive pricing, can capitalize on customer demand and drive overall sales.

**Personalized offers for high spenders:** Identifying customers with high total spending, such as males or customers in specific age groups, allows for targeted marketing and personalized offers. Providing exclusive discounts, loyalty rewards, or special privileges to these customers can encourage repeat purchases and increase customer satisfaction.

**Implement loyalty program:** Implementing a loyalty program that offers incentives, rewards, and exclusive deals to encourage repeat purchases and increase customer retention. Targeted loyalty programs can be designed for male customers, single customers, and customers in specific age groups.

**Enhance product offerings:** Analyze the popular product categories and identify opportunities to expand the product range within those categories. This can attract more customers and increase sales. Additionally, identify complementary products or cross-selling opportunities to encourage customers to make additional purchases.

**Customer engagement:** Implement targeted marketing campaigns and communication strategies to engage customers regularly. This can include personalized email campaigns, social media engagement, and special promotions tailored to different customer segments. Keeping customers informed about new products, offers, and events can increase their engagement and encourage them to make more purchases.

**Collaborations and partnerships:** Explore collaborations with popular brands or influencers that resonate with the target customer segments. These collaborations can help attract new customers, create buzz, and increase brand visibility. It can also provide opportunities for joint promotions or exclusive offers.

**Seasonal and event-based promotions:** Leverage seasonal events, holidays, and special occasions to offer targeted promotions and discounts. Aligning marketing campaigns and product offerings with these events can create a sense of urgency and drive sales.

**Customer feedback and reviews:** Actively seek feedback from customers to understand their

preferences, pain points, and suggestions for improvement. Encourage customers to leave reviews and ratings to build social proof and credibility. Utilize this feedback to make necessary improvements and refine the customer experience.

**Personalization and customization:** Invest in technology and data analytics to provide personalized recommendations, product suggestions, and customized offers based on individual customer preferences and past purchase history. This level of personalization can enhance the customer experience and increase conversion rates.

**Competitive pricing and promotions:** Continuously monitor competitors' pricing and promotional activities to ensure competitiveness. Offer price-match guarantees or price comparison tools to instill confidence in customers that they are getting the best value for their purchases.